



Teoria e Aplicações de Inteligência Computacional I

Unidade 1 – Aula 3 – Introdução aos Fundamentos de
Inteligência Computacional

Prof. Dr. Guilherme Augusto Defalque

Mestrado em Engenharia Elétrica - PPGEE - UFMS

Conceitos e paradigmas de redes neurais

Agenda da Aula



Cap. 1 — Introdução às Redes Neurais Artificiais

Perceptron · MLP · Sigmoides



Cap. 2 — Treinamento de Redes

Backpropagation · Gradiente Descendente

3

Cap. 3 – Hiperparâmetros

Modos de treinamento e Hiperparâmetros do modelo

4

Cap. 4 – Funções de Ativação

Evitar que o gradiente "morra" e deixe de atualizar os pesos

5

Cap. 5 – Implementação Prática em Python

ANN com Pytorch



Teoria e Aplicações de Inteligência Computacional I

Unidade 1 – Aula 3 – Introdução aos Fundamentos de
Inteligência Computacional

Capítulo 1 – Introdução às Redes Neurais Artificiais

Perceptron · MLP · Sigmoides

Inspiração Biológica: O Neurônio Biológico

O cérebro humano, com aproximadamente **10 bilhões de neurônios** e **60 trilhões de sinapses**, opera com notável **eficiência energética** ($\sim 10^{-16}$ joules/operação). Sua **plasticidade cerebral** — a capacidade de criar e modificar conexões sinápticas — serve como análogo fundamental para o **aprendizado em Redes Neurais Artificiais**.



Dendritos

Zonas receptivas que recebem sinais de milhares de outras células. Equivalem às entradas $(x_1, x_2, \dots, x_n)$ do neurônio artificial.



Corpo Celular (Soma)

Integra e processa todos os sinais recebidos pelos dendritos. Equivale à soma ponderada + bias: $v_k = \sum w_{kj} \cdot x_j + b_k$



Axônio

"Linha de transmissão" que propaga o sinal de saída do neurônio para outras células. Equivale à saída y_k do modelo.



Sinapses

Junções que convertem sinais elétricos em químicos (e vice-versa), modulando a força da conexão. Equivalem aos pesos sinápticos w_{kj} .



Potenciais de Ação (Spikes)

Pulsos de voltagem que disparam quando o estímulo ultrapassa um limiar. Equivalem à função de ativação $\phi(v_k)$, que decide se o neurônio "dispara".

Componentes do Neurônio Artificial

Cada neurônio artificial processa informações através de uma série de etapas bem definidas.

01

Entradas ($x_1, x_2, \dots, x_m$)

Sinais do ambiente ou de outras camadas. Ex: medições de tensão/corrente.

02

Pesos Sinápticos (w_{kj})

Força de cada conexão. Positivos = excitatórios. Negativos = inibitórios. Armazenam o conhecimento.

03

Bias (b_k)

Parâmetro externo que ajusta o "limiar de disparo". Tratado como peso extra $w_{k0} = b_k$ com entrada fixa $x_0 = +1$.

04

Soma Ponderada

$$u_k = \sum w_{kj}x_j$$

05

Campo Local Induzido

$$b_k + \sum w_{kj}x_j$$

(com $x_0 = 1$)

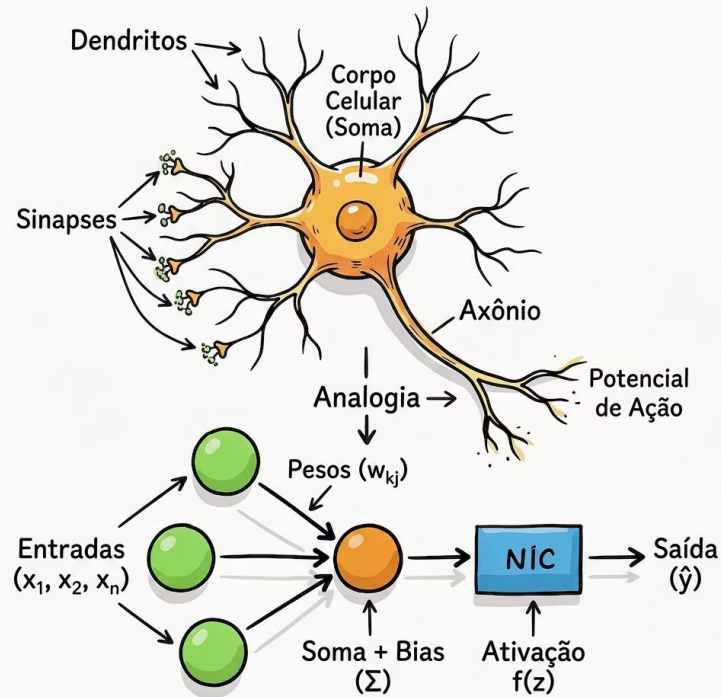
06

Função de Ativação

$$y_k = \varphi(v_k)$$

Limita a amplitude da saída e introduz não-linearidade.

Do Neurônio Biológico ao Artificial



Dendritos →
Entradas ($x_1 \dots x_n$)

Sinais recebidos do
ambiente.

Sinapses → Pesos
(w_{kj})

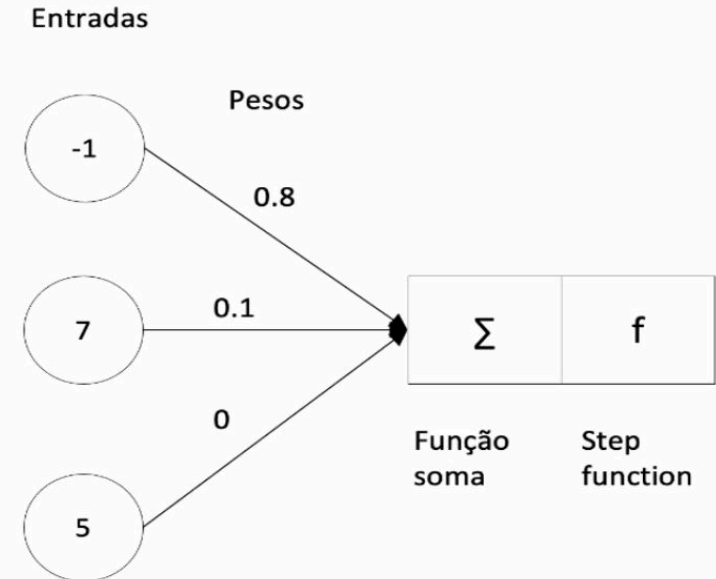
Força de cada
conexão.

Corpo Celular →
Soma + Bias (z)

$$v_k = \sum w_{kj} x_j + b_k$$

Potencial de Ação
→ Ativação $f(z)$

$$y_k = \varphi(v_k)$$



Axônio → Saída (y)

Propaga o resultado para a próxima camada.

Exemplo Numérico: Calculando um Neurônio Artificial

Vamos calcular manualmente a saída de um neurônio com 3 entradas, usando a função de ativação degrau.

01

Dados de Entrada

Entradas: $x_1 = 1$, $x_2 = 0$, $x_3 = 1$

Pesos: $w_1 = 0,5$ | $w_2 = -1,0$ | $w_3 = 0,8$

Bias: $b = -0,2$

02

Soma Ponderada (z)

Calcular a soma ponderada de cada entrada multiplicada pelo seu peso:

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

$$z = (0,5 \times 1) + (-1,0 \times 0) + (0,8 \times 1) + (-0,2)$$

$$z = 0,5 + 0 + 0,8 - 0,2 = 1,1$$

03

Função de Ativação (Degrâu)

Aplicar a função degrau ao resultado $z = 1,1$:

$$f(z) = \begin{cases} 1, & \text{se } z \geq 0 \\ 0, & \text{se } z < 0 \end{cases}$$

Como $z = 1,1 \geq 0 \rightarrow f(z) = 1$

04

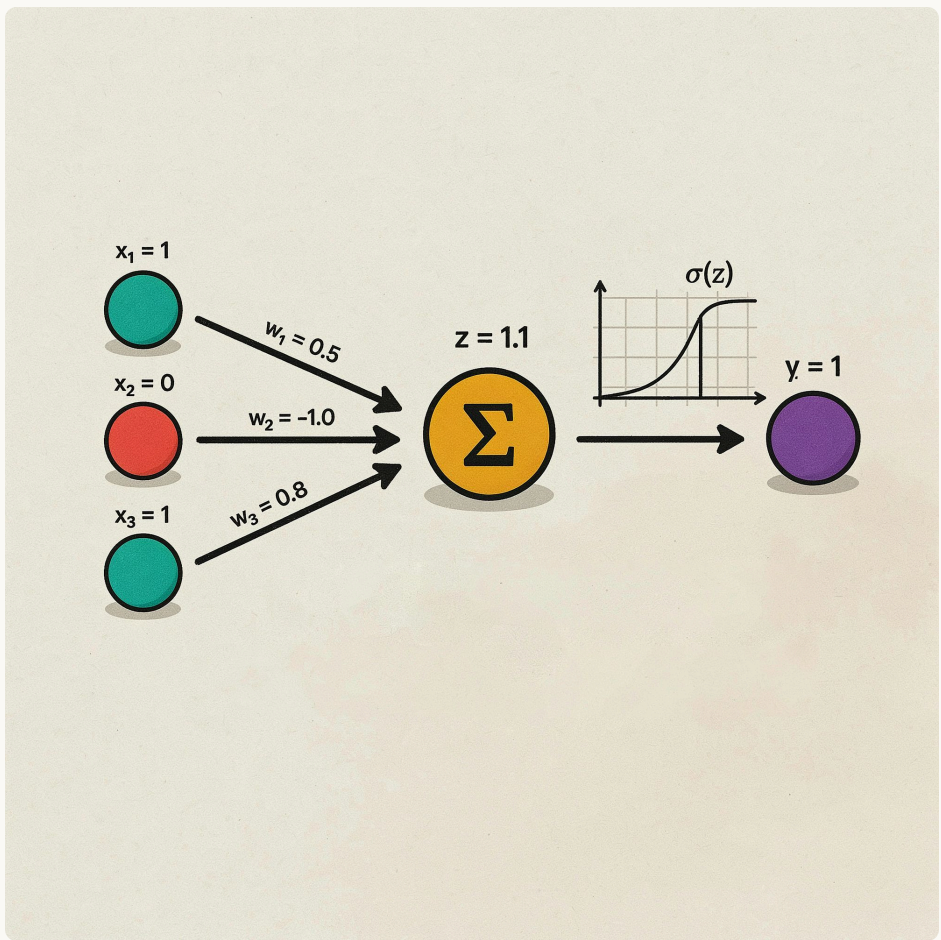
Saída ($\hat{y}$)

O neurônio dispara! A saída é $\hat{y} = 1$.

Interpretação: com esses pesos e entradas, o neurônio ativou — equivalente ao "potencial de ação" no neurônio biológico.

Resumo: Neurônio Artificial em Ação

Componente	Símbolo	Valor
Entradas	x_1, x_2, x_3	1, 0, 1
Pesos	w_1, w_2, w_3	0,5 / -1,0 / 0,8
Bias	b	-0,2
Soma ponderada	z	$0,5 + 0 + 0,8 - 0,2 = 1,1$
Função de ativação	$f(z)$	Degrau
Saída	$\hat{y}$	1 ✓ (neurônio disparou)



Teoria do Perceptron

O Perceptron, proposto por Frank Rosenblatt em 1958, é o modelo mais simples de neurônio artificial com capacidade de aprendizado. Sua teoria formaliza as condições e garantias do processo de treinamento.

Regra de Aprendizagem do Perceptron

$$w_j(n + 1) = w_j(n) + \eta \cdot e(n) \cdot x_j(n)$$

Onde: η = taxa de aprendizagem, $e(n) = y_{\text{real}} - \hat{y}(n)$ = erro na iteração n , x_j = j -ésima entrada

Teorema da Convergência do Perceptron

Se os dados de treinamento forem **linearmente separáveis**, o algoritmo do Perceptron é garantido de convergir em um número finito de iterações, encontrando um hiperplano separador.

Função de Ativação Degrau (Heaviside)

$$\hat{y} = f(z) = \begin{cases} 1 & \text{se } z \geq \theta \\ 0 & \text{se } z < \theta \end{cases}$$

Onde $z = \sum_j w_j x_j + b$ (combinação linear das entradas)

Limitação Fundamental

O Perceptron de camada única **não consegue resolver problemas não linearmente separáveis**, como o XOR. Essa limitação motivou o desenvolvimento do MLP (Multilayer Perceptron).

Porta AND: Treinamento do Perceptron

O perceptron aprende os pesos iterativamente. Iniciamos com $w_1=0$, $w_2=0$ e $b=0$, taxa de aprendizagem $\eta=1$, e aplicamos a regra de atualização a cada amostra.

$$w_j(n + 1) = w_j(n) + \eta \cdot x_j \cdot e(n) \quad \text{onde} \quad e(n) = y_{real} - \hat{y}$$

Tabela Verdade AND

x_1	x_2	y_{real}
0	0	0
0	1	0
1	0	0
1	1	1

Estado Inicial

$w_1 = 0, w_2 = 0, b = 0, \eta = 1$

01

Amostra (0,0) $\rightarrow y_{real} = 0$

$z = 0 \cdot 0 + 0 \cdot 0 + 0 = 0 \rightarrow f(z) = 1$ (degrau: $z \geq 0 \rightarrow 1$)

$e = 0 - 1 = -1$

$w_1=0 \mid w_2=0 \mid b=-1 \checkmark$

03

Amostra (1,0) $\rightarrow y_{real} = 0$

$z = 0 \cdot 1 + 0 \cdot 0 + (-1) = -1 \rightarrow f(z) = 0$

$e = 0 - 0 = 0 \rightarrow$ sem atualização

$w_1=0, w_2=0, b=-1$ (sem mudança)

02

Amostra (0,1) $\rightarrow y_{real} = 0$

$z = 0 \cdot 0 + 0 \cdot 1 + (-1) = -1 \rightarrow f(z) = 0$

$e = 0 - 0 = 0 \rightarrow$ sem atualização

$w_1=0, w_2=0, b=-1$ (sem mudança)

04

Amostra (1,1) $\rightarrow y_{real} = 1$

$z = 0 \cdot 1 + 0 \cdot 1 + (-1) = -1 \rightarrow f(z) = 0$

$e = 1 - 0 = 1$

$w_1=1 \mid w_2=1 \mid b=0 \checkmark$

$\rightarrow$ **Pesos ao final da Época 1: $w_1=1, w_2=1, b=0$**

Porta AND: Época 2 e Convergência

Continuando com os pesos obtidos na Época 1: $w_1=1$, $w_2=1$, $b=0$. Verificamos se o perceptron já aprendeu corretamente.

01

Amostra (0,0) $\rightarrow y_{\text{real}} = 0$

$z = 1 \cdot 0 + 1 \cdot 0 + 0 = 0 \rightarrow f(z) = 1$ (degrau: $z \geq 0 \rightarrow 1$)

$e = 0 - 1 = -1$

$w_1 = 1 + 1 \cdot 0 \cdot (-1) = 1 \mid w_2 = 1 + 1 \cdot 0 \cdot (-1) = 1 \mid b = 0 + 1 \cdot (-1) = -1$
(atualização dos pesos)

03

Amostra (1,0) $\rightarrow y_{\text{real}} = 0$

$z = 1 \cdot 1 + 0 \cdot 0 + (-2) = -1 \rightarrow f(z) = 0$

$e = 0 - 0 = 0 \rightarrow$ sem atualização

$w_1 = 1, w_2 = 0, b = -2$

O processo continua...

O perceptron continua iterando até que nenhuma amostra gere erro ($e=0$ para todas).

02

Amostra (0,1) $\rightarrow y_{\text{real}} = 0$

$z = 1 \cdot 0 + 1 \cdot 1 + (-1) = 0 \rightarrow f(z) = 1$ (degrau: $z \geq 0 \rightarrow 1$)

$e = 0 - 1 = -1$

$w_1 = 1 + 1 \cdot 0 \cdot (-1) = 1 \mid w_2 = 1 + 1 \cdot 1 \cdot (-1) = 0 \mid b = -1 + 1 \cdot (-1) = -2$
(atualização dos pesos)

04

Amostra (1,1) $\rightarrow y_{\text{real}} = 1$

$z = 1 \cdot 1 + 0 \cdot 1 + (-2) = -1 \rightarrow f(z) = 0$

$e = 1 - 0 = 1$

$w_1 = 1 + 1 \cdot 1 \cdot 1 = 2 \mid w_2 = 0 + 1 \cdot 1 \cdot 1 = 1 \mid b = -2 + 1 \cdot 1 = -1$
(atualização dos pesos)

$\rightarrow$ Pesos ao final da Época 2: $w_1=2, w_2=1, b=-1$

Verificação Final (pesos convergidos)

Com pesos finais convergidos (ex: $w_1=1, w_2=1, b=-1,5$), testar todas as amostras: (0,0): $z=-1,5 \rightarrow 0 \checkmark \mid$ (0,1): $z=-0,5 \rightarrow 0 \checkmark \mid$ (1,0): $z=-0,5 \rightarrow 0 \checkmark \mid$ (1,1): $z=0,5 \rightarrow 1 \checkmark$

Exercício: Treine o Perceptron para a Porta OR

Com base no exemplo da Porta AND, treine o perceptron para implementar a função lógica OR. Use os mesmos parâmetros iniciais: $w_1=0$, $w_2=0$, $b=0$ e taxa de aprendizagem $\eta=1$.

Tabela Verdade OR

x_1	x_2	y_{real}
0	0	0
0	1	1
1	0	1
1	1	1

Estado Inicial

$w_1 = 0, w_2 = 0, b = 0, \eta = 1$

Função de ativação: Degrau

$f(z) = 1$ se $z \geq 0$, senão 0

Passo 1 — Época 1, Amostra (0,0)

Calcule $z = w_1 \cdot 0 + w_2 \cdot 0 + b$. Aplique $f(z)$.
Calcule $e = y_{\text{real}} - \hat{y}$. Atualize w_1 , w_2 e b conforme a regra de aprendizagem.

Passo 2 — Época 1, Amostra (0,1)

Use os pesos atualizados. Calcule $z = w_1 \cdot 0 + w_2 \cdot 1 + b$. Aplique $f(z)$. Calcule e e atualize os pesos.

Passo 3 — Época 1, Amostra (1,0)

Use os pesos atualizados. Calcule $z = w_1 \cdot 1 + w_2 \cdot 0 + b$. Aplique $f(z)$. Calcule e e atualize os pesos.

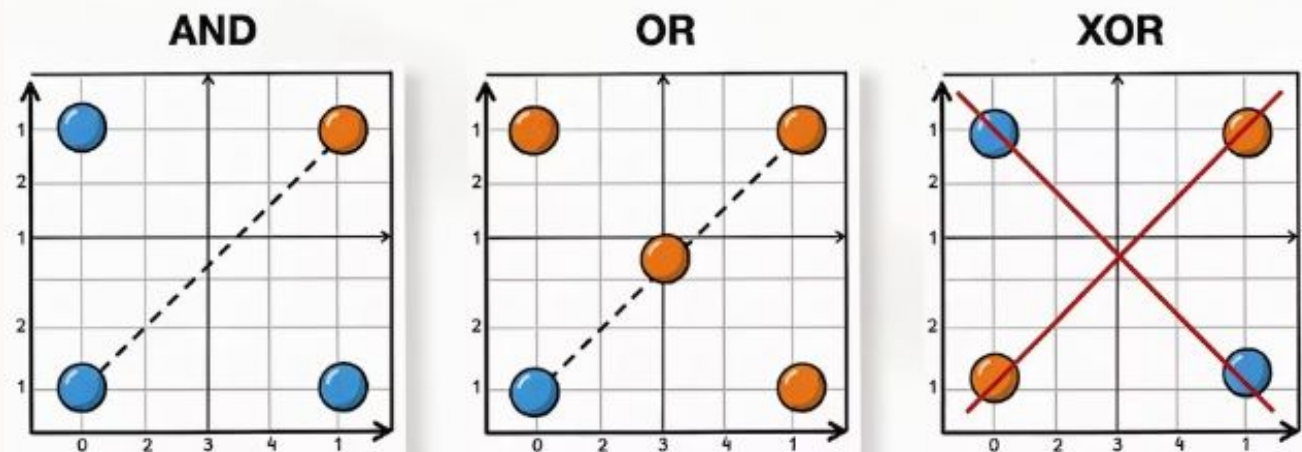
Passo 4 — Época 1, Amostra (1,1)

Use os pesos atualizados. Calcule $z = w_1 \cdot 1 + w_2 \cdot 1 + b$. Aplique $f(z)$.
Calcule e e atualize os pesos. **Quais são os pesos ao final da Época 1?**

Continue até Convergir

Repita o processo para as épocas seguintes. **Dica:** a função OR é linearmente separável — o Teorema da Convergência do Perceptron garante que o algoritmo converge! 🎯

Separabilidade Linear: AND, OR e XOR



AND ✓

Os três pontos com saída 0 ficam de um lado e o único ponto com saída 1 (1,1) do outro. Uma única reta separa as classes.

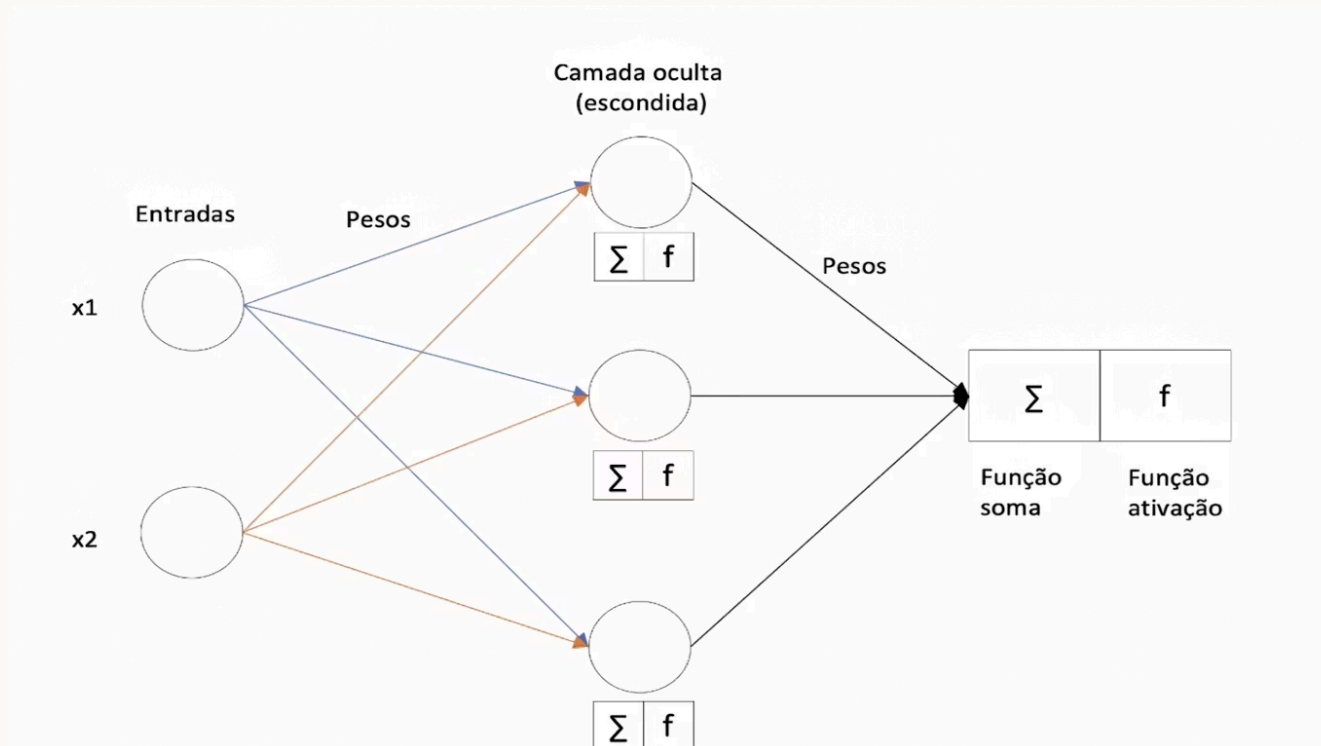
OR ✓

O único ponto com saída 0 (0,0) fica isolado de um lado. Os três pontos com saída 1 ficam do outro. Uma reta resolve.

XOR ✗

Os pontos com saída 1 estão em (0,1) e (1,0) — diagonalmente opostos aos zeros. Nenhuma reta consegue separar as classes. Exige MLP!

Multilayer Perceptron (MLP)



Entradas (x_1, x_2)

Cada entrada é conectada a todos os neurônios da camada oculta por meio de pesos. Cada conexão tem um peso diferente, que é ajustado durante o treinamento.

Camada Oculta (Escondida)

Cada neurônio oculto realiza: soma ponderada das entradas (Σ) seguida de uma função de ativação f . Permite aprender representações não lineares dos dados — o que o perceptron simples não consegue.

$$h_j = f\left(\sum_i w_{ij}x_i + b_j\right)$$

Pesos entre Camadas

Cada camada possui sua própria matriz de pesos. Os pesos são ajustados pelo algoritmo de Backpropagation para minimizar o erro da saída.

MLP: Camada de Saída e Poder de Representação

Continuando a análise do MLP — como a camada de saída processa os sinais e por que múltiplas camadas são essenciais para problemas não lineares.

Camada de Saída

Recebe as saídas da camada oculta (h_1, h_2, h_3), realiza nova soma ponderada (Σ) e aplica função de ativação f para produzir a saída final $\hat{y}$.

$$\hat{y} = f\left(\sum_j w_j h_j + b\right)$$

A função de ativação da saída depende do problema: sigmoide para classificação binária, softmax para multiclasse, linear para regressão.

Por que "Multi" Camadas?

Com pelo menos uma camada oculta, o MLP consegue resolver problemas não linearmente separáveis (como XOR), aproximando qualquer função contínua — Teorema da Aproximação Universal (Cybenko, 1989).

$$\text{MLP} \approx f : \mathbb{R}^n \rightarrow \mathbb{R}^m \quad (\text{qualquer função contínua})$$

Quanto mais camadas e neurônios, maior a capacidade de representação — mas também o risco de overfitting.

Entrada $\rightarrow$ Dados brutos $x_1, x_2, \dots, x_n$

Camada(s) Oculta(s) $\rightarrow$ Transformações não lineares: $h = f(Wx + b)$

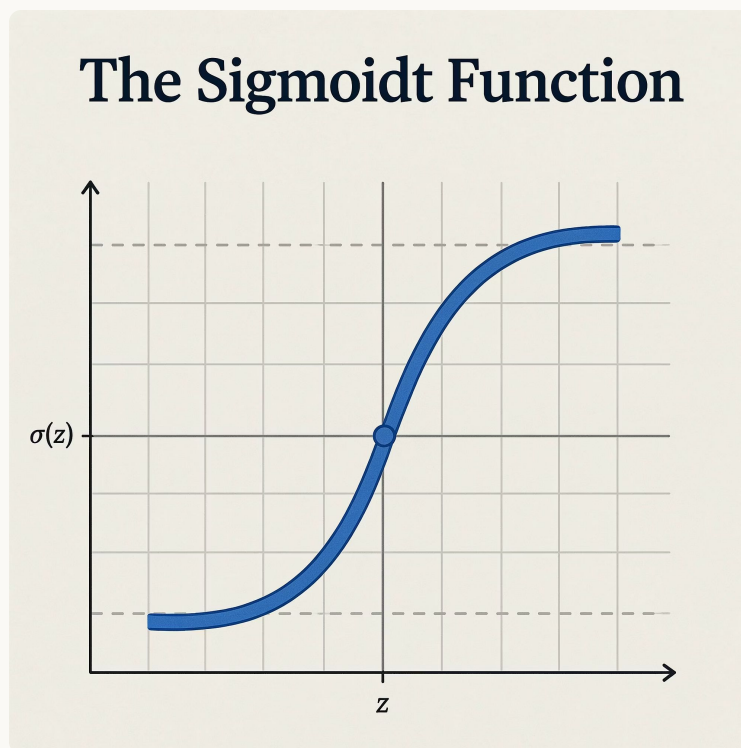
Saída $\rightarrow$ Predição final: $\hat{y} = f(Wh + b)$

A Função Sigmoid: Ativação Essencial do MLP

A função sigmoide foi a função de ativação dominante nas primeiras redes neurais e no MLP clássico. Ela transforma qualquer valor real em uma saída entre 0 e 1, interpretável como probabilidade.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

z	$\sigma(z)$
$-\infty$	0
-2	0,12
0	0,50
2	0,88
$+\infty$	1



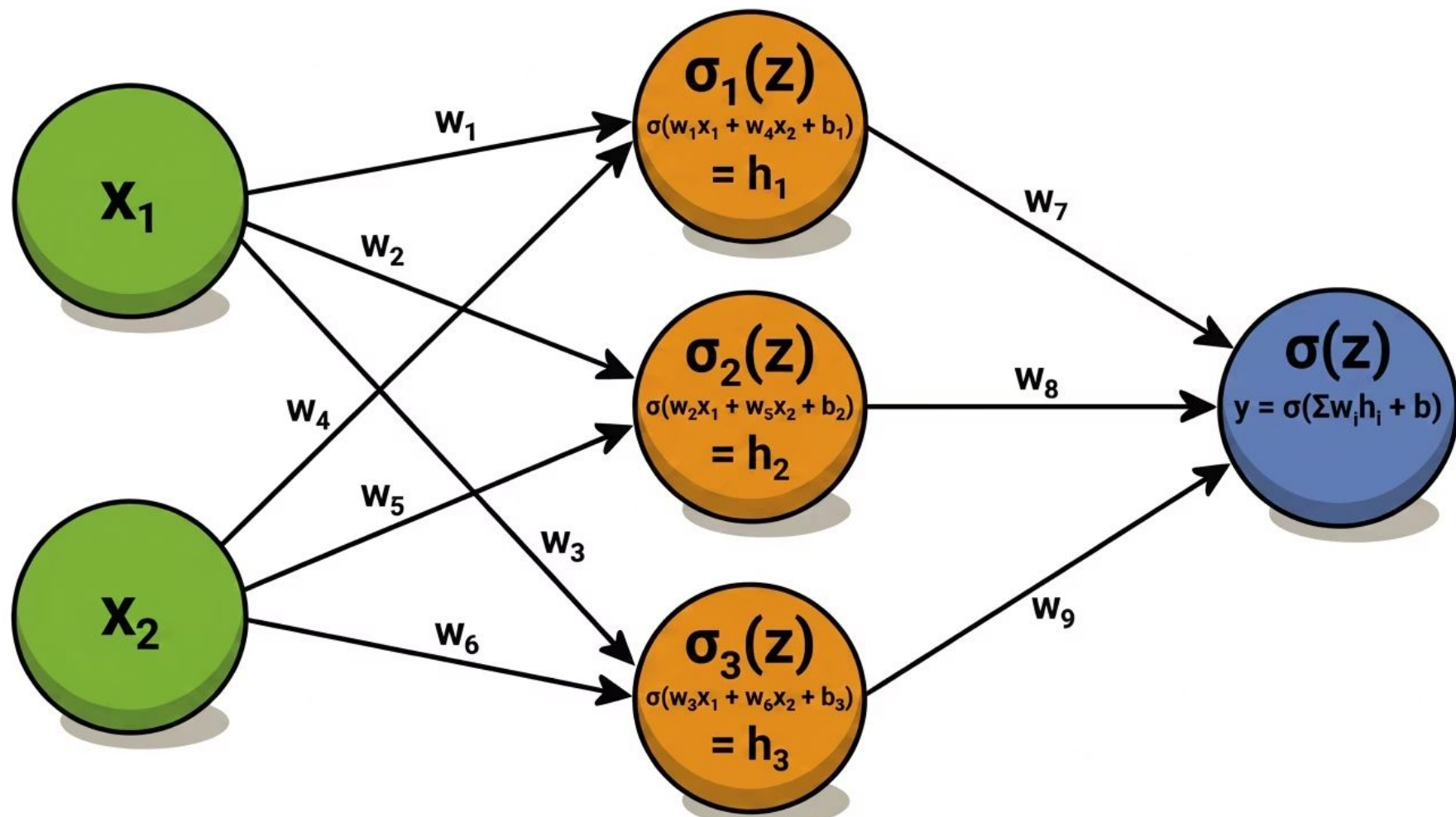
Saída entre 0 e 1

Ideal para classificação binária – interpretável diretamente como probabilidade de pertencer a uma classe.

Não Linearidade

Permite ao MLP aprender fronteiras de decisão complexas – algo impossível com funções lineares.

Arquitetura MLP: 2 Entradas, 3 Neurônios Ocultos, 1 Saída



Arquitetura MLP: Camadas e Propagação Direta

01

MLP

Composição de funções em cadeia:

$f(x) = f^3(f^2(f^1(x)))$. A profundidade é o número de camadas.

02

Camada de Entrada

Distribui os sinais de entrada; não realiza computação.

03

Camadas Ocultas

Atuam como detectores de características, transformando o espaço de entrada. "Ocultas" por não serem diretamente observáveis.

04

Camada de Saída

Gera a resposta final do modelo: rótulo para classificação ou valor contínuo para regressão.

05

Propagação Direta (Forward)

Fluxo de informação unidirecional da entrada para a saída, sem feedback.

06

Cálculo por Camada

$$h = \varphi(W^T x + b)$$

Onde h é a saída da camada, φ a função de ativação, W os pesos, x a entrada e b o bias.

MLP para XOR: Forward Pass com Sigmoid

Considere uma MLP com 2 entradas (x_1, x_2), 3 neurônios na camada oculta e 1 neurônio de saída. Usamos a sigmoide como ativação. Pesos iniciais arbitrários para ilustrar o forward pass da primeira rodada.

Pesos Iniciais (arbitrários)

Camada Oculta (W^1)

	h_1	h_2	h_3
x_1	0,5	0,3	-0,4
x_2	-0,2	0,8	0,6
b	0,1	-0,3	0,2

Camada de Saída (W^2)

	$\hat{y}$
h_1	0,7
h_2	-0,5
h_3	0,4

Exemplo: Amostra $x_1=1, x_2=1$ (XOR=0)

Passo 1 – Camada Oculta:

$$z_{h_1} = 0,5(1) + (-0,2)(1) + 0,1 = 0,4 \Rightarrow h_1 = \sigma(0,4) \approx 0,599$$

$$z_{h_2} = 0,3(1) + 0,8(1) + (-0,3) = 0,8 \Rightarrow h_2 = \sigma(0,8) \approx 0,690$$

$$z_{h_3} = -0,4(1) + 0,6(1) + 0,2 = 0,4 \Rightarrow h_3 = \sigma(0,4) \approx 0,599$$

Passo 2 – Camada de Saída:

$$z_{out} = 0,7(0,599) + (-0,5)(0,690) + 0,4(0,599) + (-0,1)$$

$$z_{out} = 0,419 - 0,345 + 0,240 - 0,1 = 0,214$$

$$\hat{y} = \sigma(0,214) \approx 0,553$$

MLP para XOR: Comparação dos Resultados (1ª Época)

Pesos iniciais aplicados às 4 amostras do XOR — resultados do forward pass:

x_1	x_2	XOR (y_{real})	$h_1=\sigma(z)$	$h_2=\sigma(z)$	$h_3=\sigma(z)$	$\hat{y}=\sigma(z)$	Erro ($y-\hat{y}$)
0	0	0	$\approx 0,525$	$\approx 0,426$	$\approx 0,550$	$\approx 0,539$	$\approx -0,539$
0	1	1	$\approx 0,475$	$\approx 0,622$	$\approx 0,690$	$\approx 0,552$	$\approx +0,448$
1	0	1	$\approx 0,646$	$= 0,500$	$\approx 0,450$	$\approx 0,545$	$\approx +0,455$
1	1	0	$\approx 0,599$	$\approx 0,690$	$\approx 0,599$	$\approx 0,553$	$\approx -0,553$

MLP para XOR: Cálculo do MSE — 1ª Época

Com base nos erros obtidos no forward pass, calculamos o MSE para avaliar o desempenho da rede na primeira época.

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

Então, para as 4 amostras do XOR:

$$\text{MSE} = \frac{1}{4} [(-0,539)^2 + (+0,448)^2 + (+0,455)^2 + (-0,553)^2]$$

$$\text{MSE} = \frac{1}{4} [0,2905 + 0,2007 + 0,2070 + 0,3058]$$

$$\text{MSE} = \frac{1}{4} \times 1,0040 \approx 0,2510$$

Interpretação do MSE $\approx 0,25$

RMSE = $\sqrt{0,25} \approx 0,50$ — a rede erra em média 0,50 por amostra.
Como os valores reais são 0 ou 1, isso equivale a um chute aleatório.
A rede ainda não aprendeu nada útil.

O que acontece a seguir?

O Backpropagation usará esses erros para calcular os gradientes e ajustar os 13 parâmetros (W^1 , W^2 , b^1 , b^2) nas épocas seguintes, reduzindo progressivamente o MSE até convergir.



Teoria e Aplicações de Inteligência Computacional I

Unidade 1 – Aula 3 – Introdução aos Fundamentos de
Inteligência Computacional

Capítulo 2 – Back-Propagation

A não-linearidade que permite às redes neurais aprender funções complexas.

Backpropagation: As Duas Fases

A backpropagation, desenvolvida em meados dos anos 1980, revolucionou o treinamento de Redes Neurais Artificiais ao resolver o complexo "problema da atribuição de crédito".

1

1. Forward Pass

- Pesos sinápticos fixos.
- Sinal propagado da entrada à saída.
- Geração da predição $\hat{y}$.

2

2. Backward Pass

- Cálculo do erro $e = d - \hat{y}$.
- Erro propagado de volta camada por camada.
- Cálculo do gradiente local δ_j de cada neurônio.

Descida do Gradiente

Minimizar $C(w_1, w_2, \dots, w_n)$ ajustando os pesos na direção oposta ao gradiente.

01

Calcular o Gradiente

Calcular $\partial E / \partial w_j$ para cada peso w_j .

02

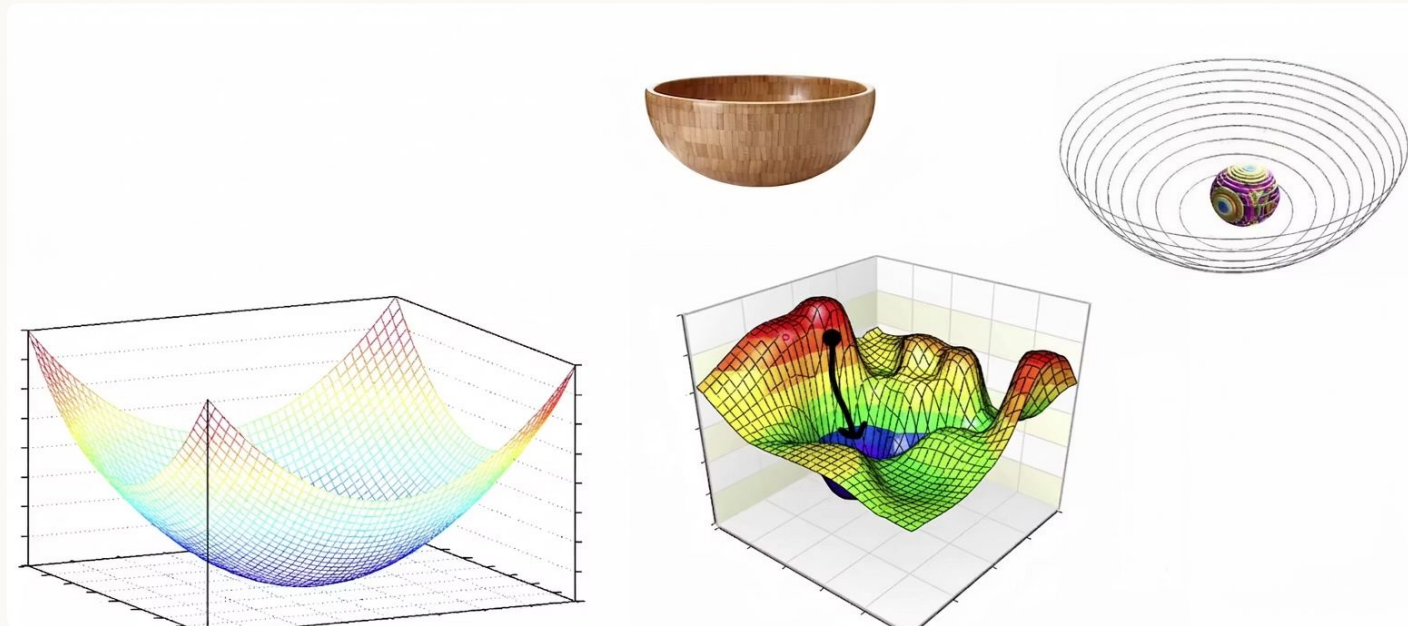
Atualizar os Pesos

Atualizar: $w_j \leftarrow w_j - \eta \cdot (\partial E / \partial w_j)$,
onde η é a taxa de aprendizado.

03

Repetir até Convergir

Repetir até o gradiente se aproximar de zero (mínimo atingido).



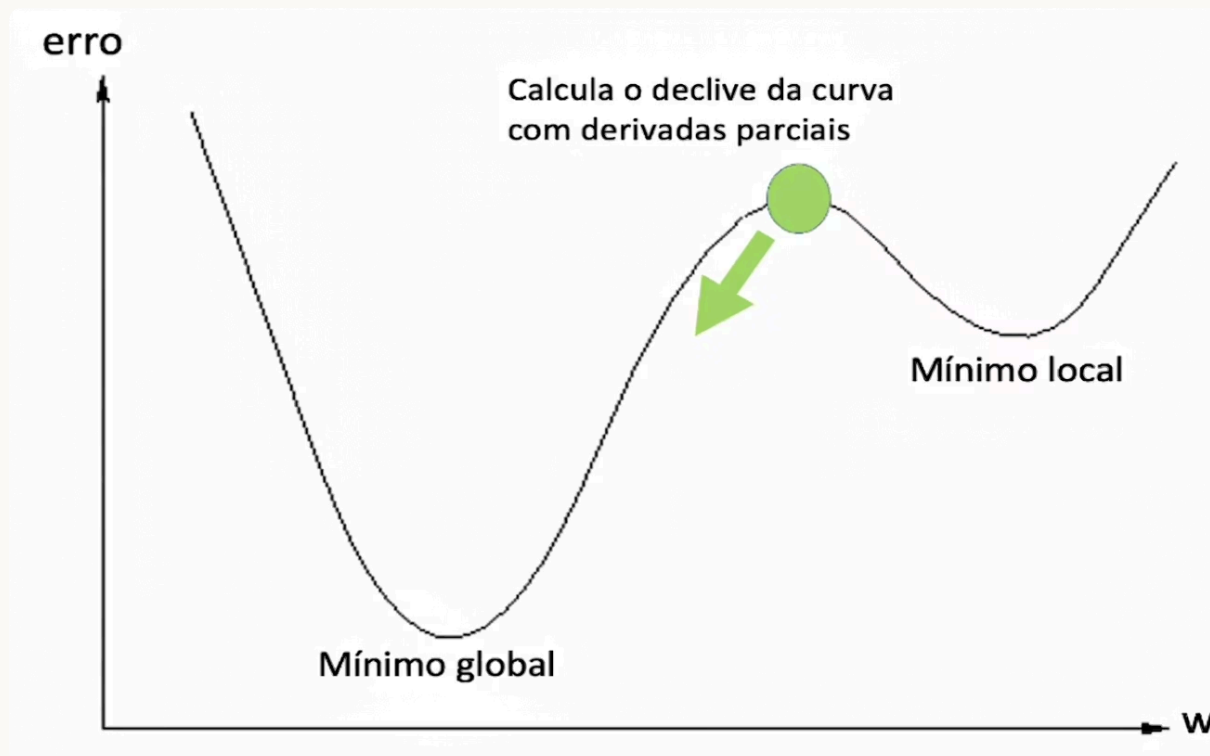
Descida do Gradiente: Mínimo Global vs. Local

Mínimo Global

O ponto mais baixo da função de custo. O objetivo do treinamento é convergir para este ponto.

Mínimo Local

Um vale secundário onde o gradiente é zero, mas não é o menor erro possível. O algoritmo pode ficar preso aqui.



Derivada Parcial: Erro e Pesos

A derivada parcial do erro em relação a cada peso indica a direção e magnitude do ajuste necessário.

O que mede?

$\partial E / \partial w_j$ indica quanto o erro **E** varia ao mudar o peso **w_j** — a sensibilidade do erro a cada peso.

$$\frac{\partial E}{\partial w_j} = \frac{\partial E}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_j}$$

O produto das duas derivas em azul é chamado de delta de saída.

Como calcular?

Aplica-se a regra da cadeia:

$\partial E / \partial w_j = \partial E / \partial \hat{y} \cdot \partial \hat{y} / \partial z \cdot \partial z / \partial w_j$, decompondo o gradiente camada a camada.

Passo 1) Cálculo dos Deltas (δ) no Backpropagation

Camada de Saída

Aplicando a regra da cadeia ao MSE: $E = \frac{1}{2}(\hat{y} - y)^2$

$$\frac{\partial E}{\partial \hat{y}} = (\hat{y} - y)$$

$$\frac{\partial \hat{y}}{\partial z} = \hat{y}(1 - \hat{y})$$

$$\delta_{out} = \frac{\partial E}{\partial z} = (\hat{y} - y) \cdot \hat{y}(1 - \hat{y})$$

Termo do erro

$(\hat{y} - y)$: derivada de $\frac{1}{2}(\hat{y} - y)^2$ em relação a $\hat{y}$.

Derivada da sigmoide

$\sigma'(z) = \hat{y} \cdot (1 - \hat{y})$: derivada da função de ativação em relação a z .

Camada Oculta

O delta de cada neurônio oculto k é propagado a partir do δ_{out} ponderado pelo peso de conexão w_k (oculto $\rightarrow$ saída):

$$\delta_{h_k} = \delta_{out} \cdot w_k \cdot h_k(1 - h_k)$$

$$\begin{cases} \delta_{h_1} = \delta_{out} \cdot w_7 \cdot h_1(1 - h_1) \\ \delta_{h_2} = \delta_{out} \cdot w_8 \cdot h_2(1 - h_2) \\ \delta_{h_3} = \delta_{out} \cdot w_9 \cdot h_3(1 - h_3) \end{cases}$$

$\delta_{out} \cdot w_k$

Propaga o erro da saída para o neurônio oculto k , ponderado pelo peso de conexão correspondente (w_7, w_8 ou w_9).

$h_k \cdot (1 - h_k)$

Derivada local da sigmoide no neurônio oculto k .

Cálculo Simplificado dos Deltas (δ)

Delta da Camada de Saída

$\delta_{saída}$

O erro da saída multiplicado pela derivada da sigmoide naquele ponto:

$$\delta_{saída} = \text{Erro} \cdot \sigma'(\hat{y}) = (y - \hat{y}) \cdot \hat{y}(1 - \hat{y})$$

Erro

$(y - \hat{y})$: diferença entre o valor real e o previsto.

$\sigma'(\hat{y})$

$\hat{y} \cdot (1 - \hat{y})$: derivada da sigmoide na saída.

Delta da Camada Oculta

δ_{oculto}

A derivada da sigmoide no neurônio oculto, multiplicada pelo peso de conexão com a saída e pelo delta da saída:

$$\delta_{h_k} = \sigma'(h_k) \cdot w_k \cdot \delta_{saída} = h_k(1 - h_k) \cdot w_k \cdot \delta_{saída}$$

$\sigma'(h_k)$

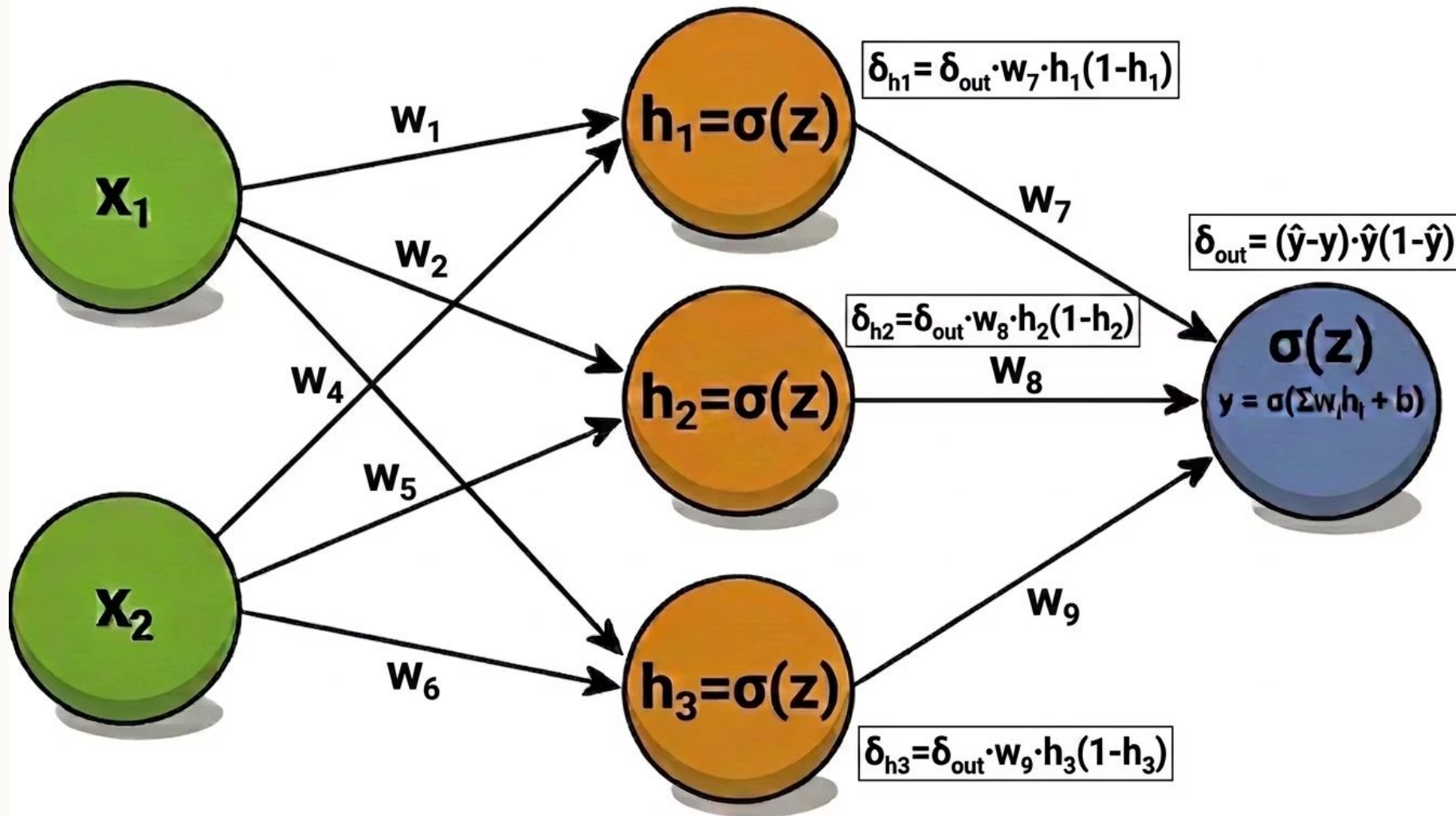
$h_k \cdot (1 - h_k)$: derivada da sigmoide no neurônio oculto k.

$w_k \cdot \delta_{saída}$

Propaga o erro da saída para o neurônio oculto k via peso de conexão w_k .

Após calcular os deltas, a atualização de cada peso é: $w_j \leftarrow w_j + \eta \cdot \delta \cdot x_j$

Deltas no Backpropagation



Passo 2) Atualização dos Pesos

Camada de Saída (W^2)

Cada peso entre a camada oculta e a saída é atualizado usando o $\delta_{saída}$ e a ativação h_k :

$$w_k \leftarrow w_k + \eta \cdot \delta_{saída} \cdot h_k$$

η (eta)

Taxa de aprendizado: controla o tamanho do passo na atualização.

h_k

Ativação do neurônio oculto k: a entrada que chegou ao peso w_k .

Camada Oculta (W^1)

Cada peso entre as entradas e o neurônio oculto k é atualizado usando o δ_{h_k} e a entrada x_j :

$$w_{kj} \leftarrow w_{kj} + \eta \cdot \delta_{h_k} \cdot x_j$$

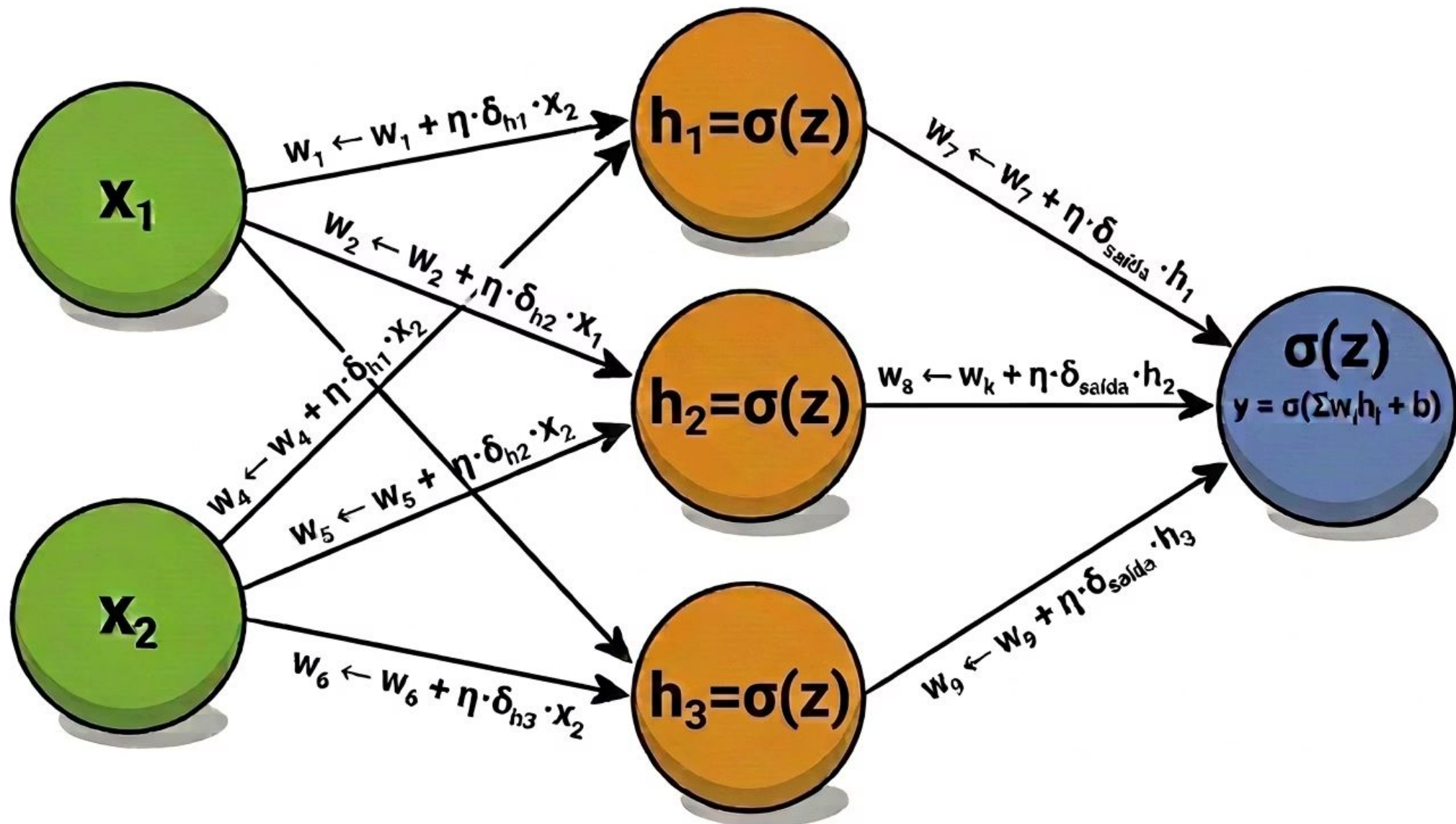
δ_{h_k}

Delta do neurônio oculto k, propagado a partir do $\delta_{saída}$.

x_j

Valor da entrada j: a entrada que chegou ao peso w_{kj} .

O processo repete para todas as amostras e todas as épocas até o erro convergir. O sinal de + assume que δ já carrega o sinal do erro ($y - \hat{y}$).





Teoria e Aplicações de Inteligência Computacional I

Unidade 1 – Aula 3 – Introdução aos Fundamentos de
Inteligência Computacional

Capítulo 3 – Modos de treinamento e Hiperparâmetros
do modelo

A não-linearidade que permite às redes neurais aprender funções complexas.

Otimização: Hiperparâmetros

Definição: São configurações externas ao modelo que não são adaptadas pelo algoritmo de aprendizado em si, mas controlam seu comportamento;

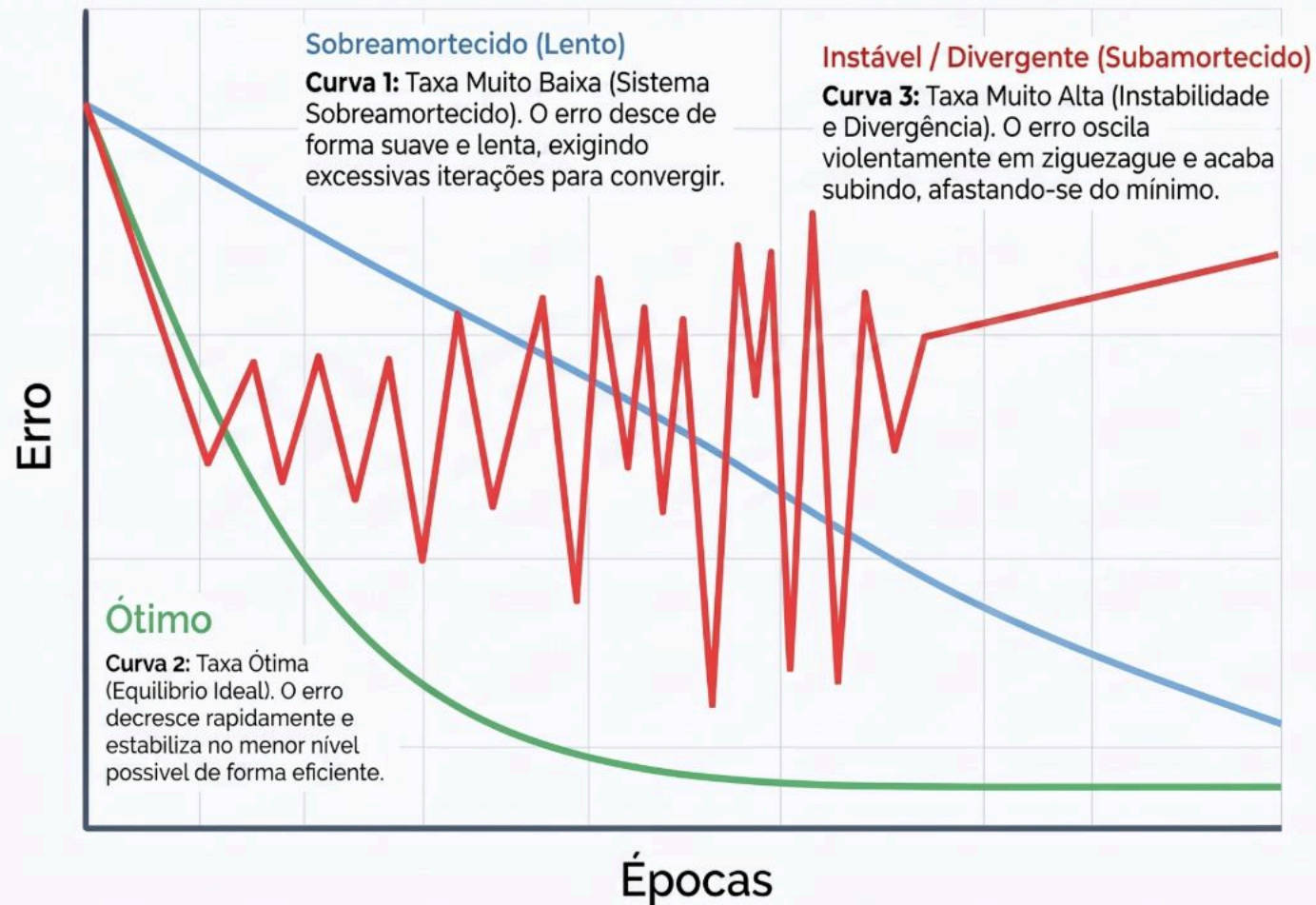
Objetivos: Ajustar a **capacidade efetiva** do modelo para encontrar o equilíbrio ideal no dilema Viés-Variância;

A Taxa de Aprendizado (η): É frequentemente considerada o **hiperparâmetro mais importante**:

- **Comportamento:**
 - **Muito baixa:** Treinamento extremamente lento e risco de ficar preso em mínimos locais pobre.
 - **Muito alta:** Causa oscilações violentas ou divergência, onde o erro aumenta em vez de diminuir.
 - **Estratégia de Annealing:** Começar com uma taxa alta e diminuí-la gradualmente para otimizar a convergência final

Otimização: Hiperparâmetros – Taxa de Aprendizizado (η)

Taxa de Aprendizizado: O Equilíbrio da Convergência



Diagnóstico do Treinamento

Overdamped (Sobreamortecido)

Ocorre quando passos minúsculos impedem a rede de chegar ao mínimo em tempo útil.

Underdamped & Instabilidade

Passos largos fazem o modelo "saltar" sobre o mínimo, gerando oscilações e divergência.

O Objetivo do Ajuste

Encontrar o maior η que ainda garanta uma convergência estável e rápida.

Tamanho do Batch: Gradiente Descendente e Variantes

Batch (Lote)

- Atualiza após **VER TODOS** os exemplos.
- Gradiente preciso, mas **lento para grandes datasets**.
- Cácula todos os gradientes, de todos os exemplos → calcula a média dos gradientes → Atualiza todos os pesos

SGD (Estocástico) → O que vimos

- Atualiza após **CADA** exemplo.
- Rápido, mas ruidoso ("**caminhada aleatória**").
- Bom para datasets grandes e redundantes.

Mini-Batch

- Atualiza após **GRUPOS** (32–256 exemplos).
- Padrão moderno. **Eficiente em GPU**.
- Efeito regularizador e estimativa imparcial do gradiente com variância menor.

SGD (Gradiente Estocástico) vs. Batch Gradient Descent

SGD (Gradiente Estocástico)

Usa 1 amostra por vez para atualizar os pesos.

1) Cada amostra i produz um gradiente para o peso w

$$\delta^{(i)} = \frac{\partial L^{(i)}}{\partial w}$$

$L^{(i)}$: perda (loss) da amostra i
 w : qualquer peso da rede



2) Atualização IMEDIATA do peso com esse gradiente

$$w \leftarrow w - \eta \delta^{(i)}$$

Resumo (SGD)

- Atualiza após cada amostra ($i = 1, 2, \dots, N$).
- Gradiente mais ruidoso.
- Pode oscilar, mas costuma convergir rápido.

Batch Gradient Descent

Usa todas as amostras para atualizar os pesos.

1) Cada amostra i produz um gradiente para o peso w

$$\begin{aligned} \delta^{(1)} &= \frac{\partial L^{(1)}}{\partial w} \\ \delta^{(2)} &= \frac{\partial L^{(2)}}{\partial w} \\ &\vdots \\ \delta^{(N)} &= \frac{\partial L^{(N)}}{\partial w} \end{aligned}$$

N : número total de amostras



2) Calcula a MÉDIA dos gradientes

$$\bar{\delta} = \frac{1}{N} \sum_{i=1}^N \delta^{(i)}$$



3) Atualização do peso APENAS ao final da época

$$w \leftarrow w - \eta \bar{\delta}$$

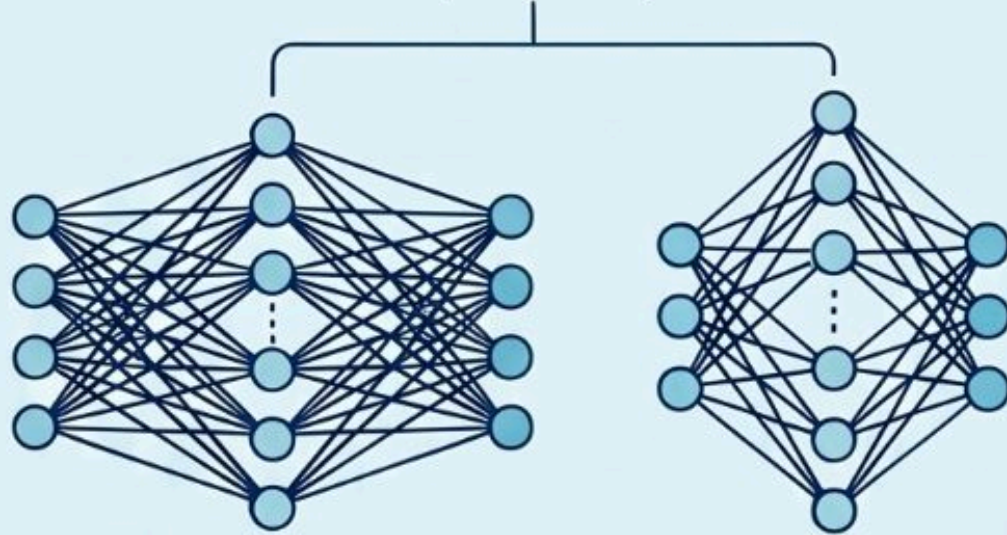
Resumo (Batch)

- Usa todas as amostras antes de atualizar.
- Gradiente mais estável (menos ruído).
- Convergência mais estável, porém pode ser mais lenta por época.

Hiperparâmetros de Arquitetura – Camadas e Neurônios

- **Número de Camadas Ocultas (Profundidade):** Camadas adicionais permitem aprender "características de características", combinando conceitos simples em abstrações complexas
- **Número de Neurônios por Camada (Largura):** Determina a capacidade de representação de cada camada; aumentar a largura aumenta a capacidade, mas eleva o custo computacional
- ***Teorema da Aproximação Universal:** Garante que uma única camada oculta com neurônios suficientes pode aproximar qualquer função contínua, mas redes mais profundas costumam ser mais eficientes.

Teorema da Aproximação Universal



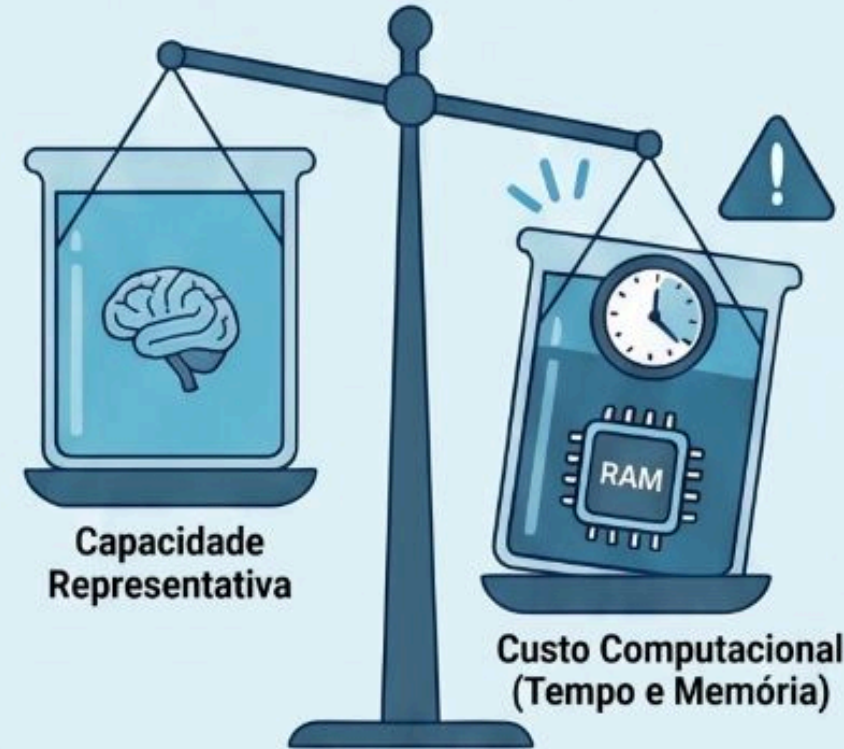
Suficiência Teórica (Rede Rasa)

Uma camada oculta pode aproximar qualquer função contínua, mas pode exigir neurônios em número exponencial.

Eficiência Prática (Rede Profunda)

Arquiteturas profundas reduzem o número total de parâmetros necessários para representar funções complexas.

Capacidade Representativa vs. Custo



Mais neurônios aumentam a capacidade do modelo, mas elevam drasticamente o uso de memória.

Mais neurônios aumentam a capacidade do modelo, mas elevam drasticamente o uso de memória.

O Termo de Momento (α)

- **Função:** Acelera o aprendizado em direções de gradiente consistente e estabiliza oscilações em direções ruidosas.
- **Analogia Física:** Age como uma "massa" que acumula velocidade (inércia) ao descer a superfície de erro. O gradiente negativo é visto como uma força que move uma partícula pelo espaço de parâmetros.
- **Benefício:** Ajuda a evitar que o processo de treinamento termine em mínimos locais rasos ou fique preso em "vales" estreitos da superfície de erro

Equação do Momento (α – Regra Delta Generalizada):

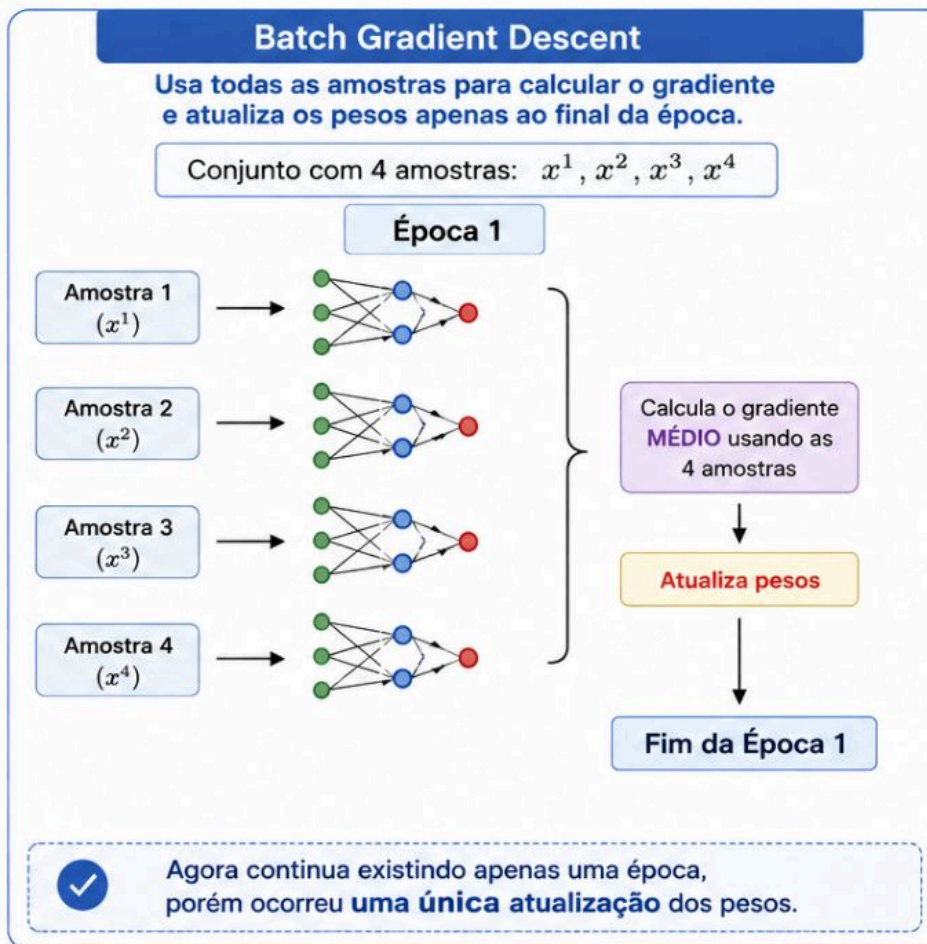
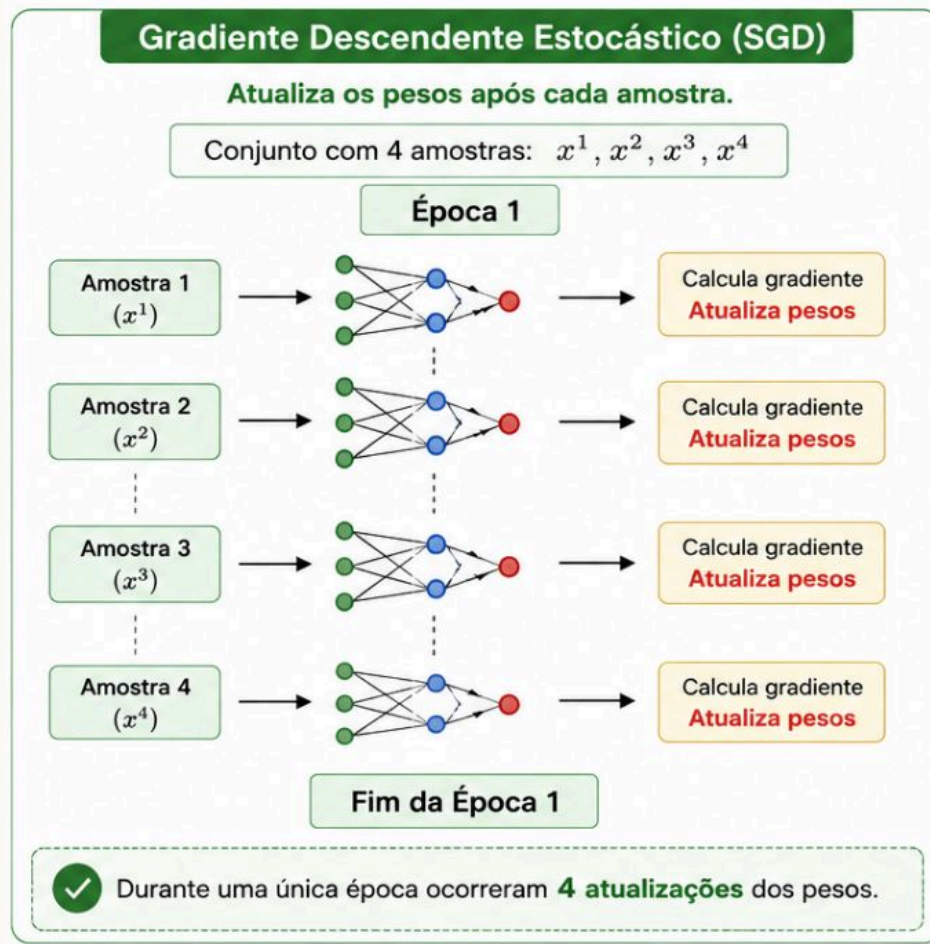
A atualização dos pesos com o termo de momento é definida por:

$$\Delta w_{ji}(n) = \alpha \Delta w_{ji}(n-1) + \eta \delta_j(n) y_i(n)$$

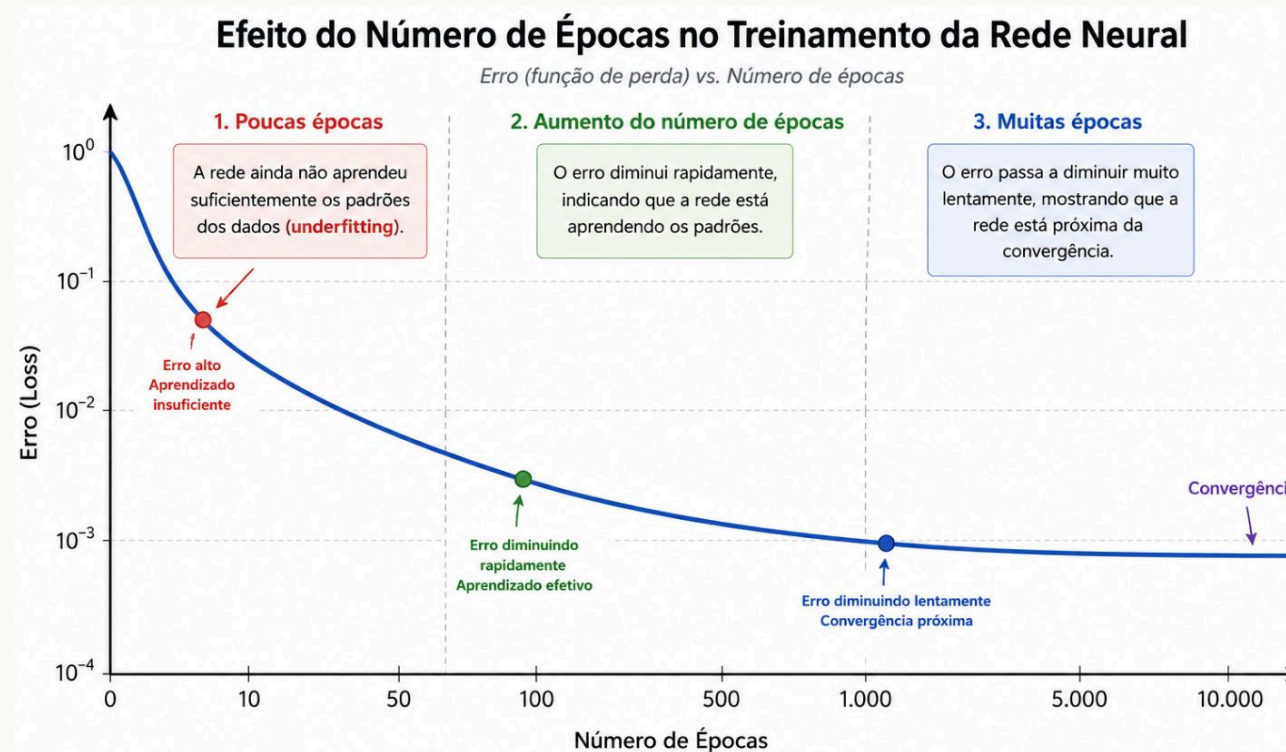
Onde:

- $\Delta w_{ji}(n)$: Ajuste atual do peso.
- α : Constante de momento ($0 \leq \alpha < 1$).
- $\Delta w_{ji}(n-1)$: Ajuste realizado na iteração anterior (memória).
- η : Taxa de aprendizado.
- $\delta_j(n)$: Gradiente local do neurônio j .
- $y_i(n)$: Sinal de entrada (ou saída da camada anterior).

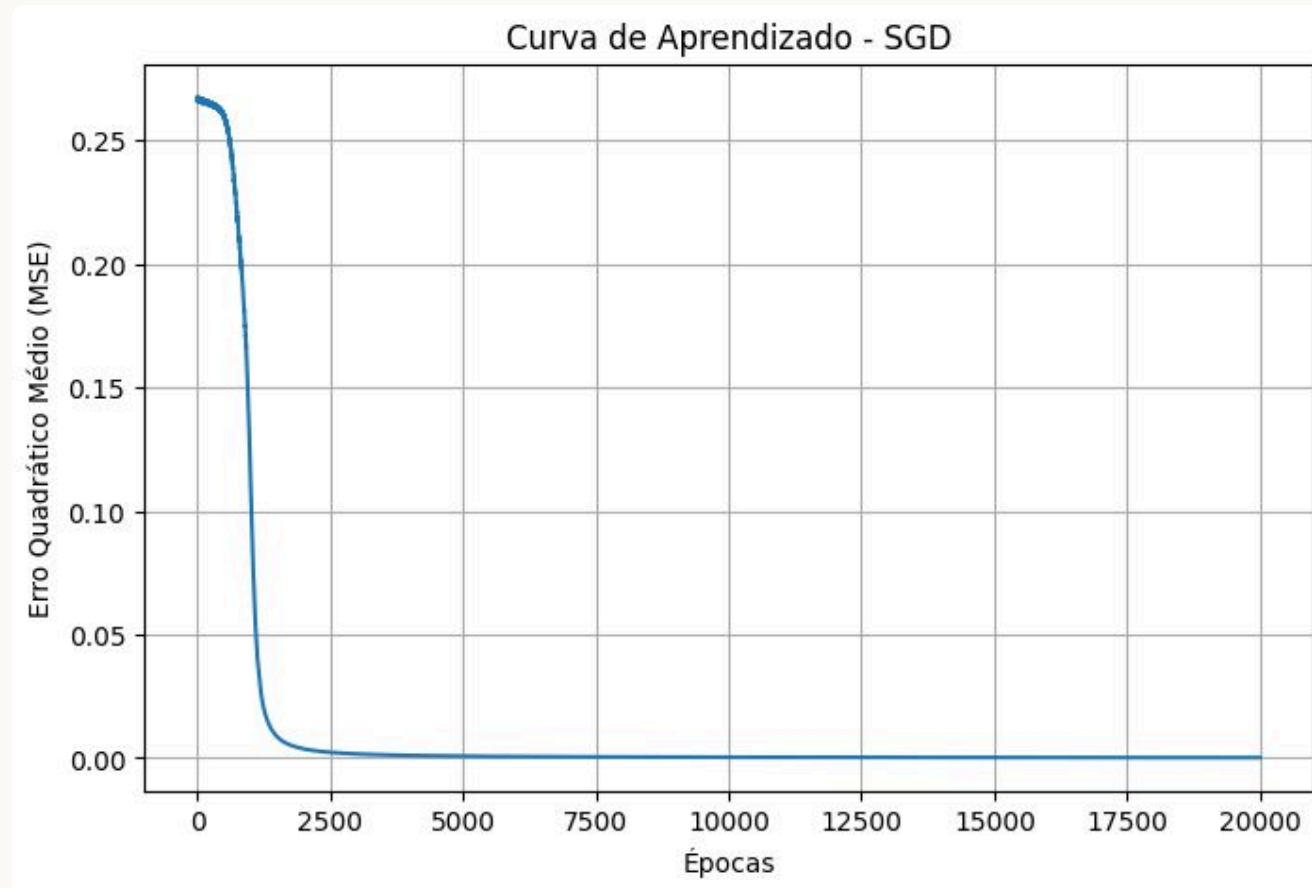
Número de Épocas (*epoch*)



Número de Épocas (*epoch*)



Número de Épocas (*epoch*)



Hiperparâmetros de regularização

Weight Decay (Decaimento de Pesos): Penaliza pesos grandes para evitar que o modelo decore ruídos (***overfitting***);

Dropout. Desativa neurônios aleatoriamente durante o treino, forçando a rede a aprender representações mais robustas e redundantes;

Early Stopping (Parada Antecipada): Monitora o erro de validação e interrompe o treino quando ele começa a subir, tratando o "tempo de treino" como um hiperparâmetro.

Weight Decay (Decaimento de Pesos)

Ideia

Durante o treinamento, alguns pesos podem crescer muito.

Quando isso acontece, a rede pode começar a se ajustar excessivamente aos dados de treinamento, aprendendo inclusive os ruídos presentes neles.

O Weight Decay adiciona uma **penalização para pesos muito grandes**.

Como funciona?

A função de custo deixa de considerar apenas o erro da rede.

Sem regularização:

$$J = \text{MSE}$$

Com Weight Decay:

$$J = \text{MSE} + \lambda \sum_i w_i^2$$

Onde:

- **MSE** representa o erro da rede
- **wi** são os pesos
- **λ** controla a intensidade da penalização
- Quanto maior o peso, maior será a penalização

Atualização dos Pesos

Sem Weight Decay:

$$w \leftarrow w - \eta \frac{\partial J}{\partial w}$$

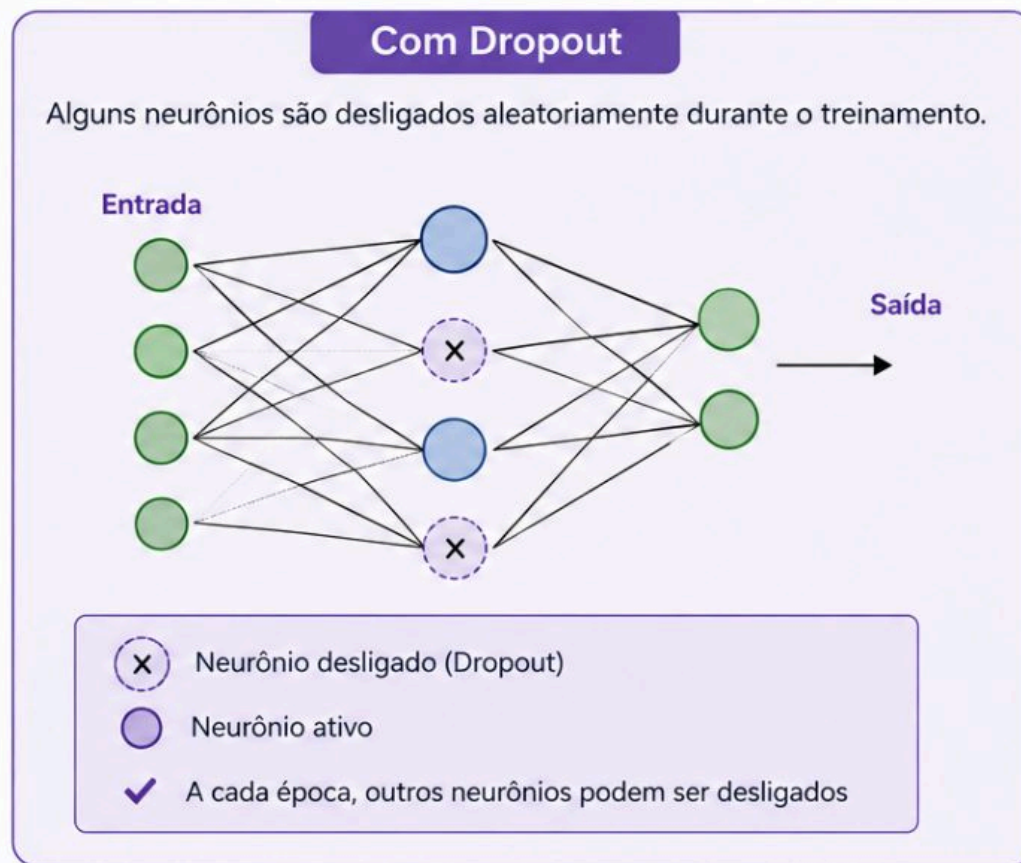
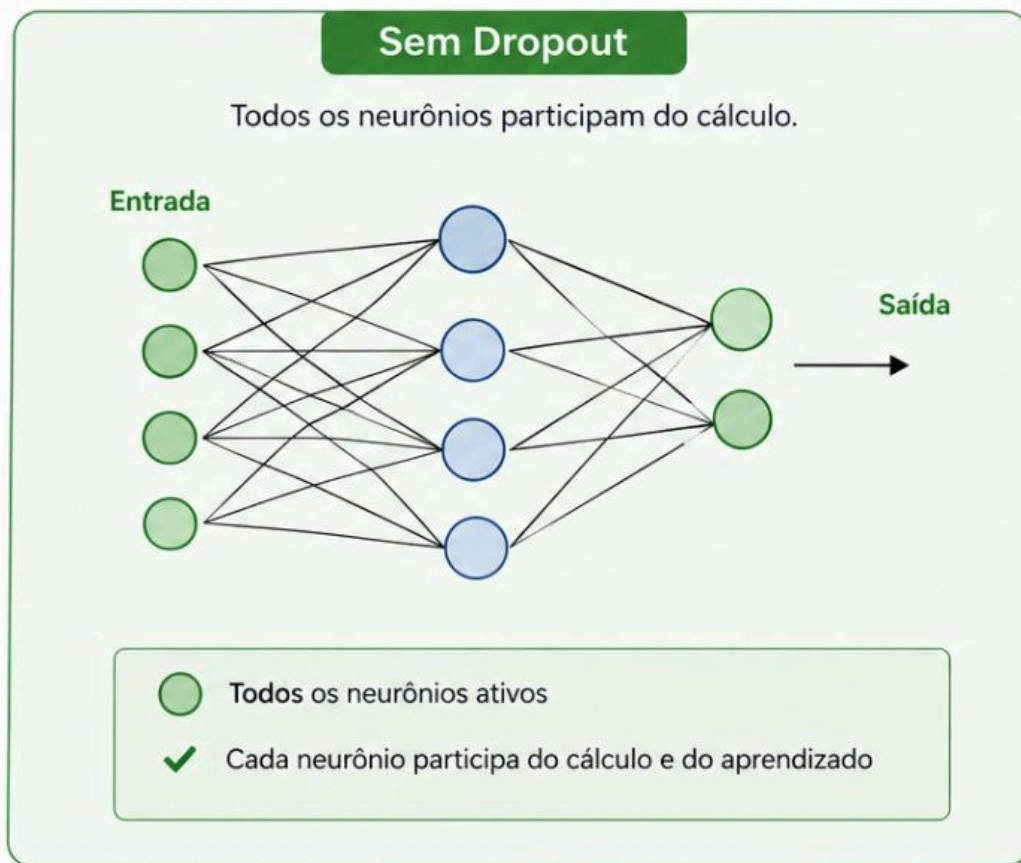
Com Weight Decay:

$$w \leftarrow w - \eta \left(\frac{\partial J}{\partial w} + \lambda w \right)$$

Observe que aparece o termo λw que **"puxa" os pesos em direção a zero**.

Dropout

Dropout é uma técnica de regularização que desativa aleatoriamente alguns neurônios durante o treinamento. Isso força a rede a não depender de neurônios específicos e a aprender representações mais robustas e gerais. **Benefícios:** ajuda a rede a generalizar melhor, evitando que ela memorize características específicas dos dados de treinamento. A cada época, uma combinação diferente de neurônios é desativada.



Separação dos dados

Conjunto de Treino: é o conjunto de dados que é separado para ajustar os pesos (parâmetros do modelo). O modelo aprender padrões a partir desses dados. Uma separação comum é 70% dos dados.

Conjunto de Validação: é usado para avaliar o desempenho do modelo durante o treinamento. Também é usado para ajustar hiperparâmetros (taxa de aprendizado, regularização, etc). Permite decidir quando para o treinamento (*Early Stopping*);

Conjunto de Teste: é usado apenas no final, depois que o modelo já está treinado e pronto. Fornece uma avaliação imparcial do desempenho final do modelo. O modelo nunca deve "ver"esses dados durante o treinamento.



Regra de ouro: Treine com o conjunto de **treino**, decida com o conjunto de **validação** e avalie com o conjunto de **teste**.

Holdout

O que é o Holdout?

O Holdout é o método mais simples de separação de dados. Consiste em dividir o conjunto de dados em partes distintas antes do treinamento, garantindo que o modelo seja avaliado em dados que nunca viu durante o aprendizado.

Como funciona?

O conjunto de dados é dividido em três partes:

01

Treino (~70 – 80%)

Usado para ajustar os pesos da rede neural.

02

Validação (~10 – 15%)

Monitorar o desempenho e ajustar hiperparâmetros durante o treinamento.

03

Teste (~10 – 15%)

Usado apenas ao final, para avaliação imparcial do modelo.

Vantagens

- Simples e rápido de implementar.
- Adequado para grandes conjuntos de dados.

Desvantagens

- A divisão pode ser tendenciosa se os dados não forem embaralhados corretamente.
- Em conjuntos pequenos, pode haver pouca representatividade em cada subconjunto.



Cross-Validation (Validação Cruzada)

O que é?

Cross-Validation é uma técnica mais robusta que o Holdout para avaliar o desempenho de um modelo. Em vez de uma única divisão fixa, os dados são divididos em k partes (folds) e o treinamento/avaliação é repetido k vezes, garantindo que todos os dados sejam usados tanto para treino quanto para validação.

Vantagens

- Todos os dados são usados para treino e validação.
- Avaliação mais confiável e menos sensível à divisão dos dados.
- Ideal para conjuntos de dados pequenos.

Desvantagens

- Computacionalmente mais custoso (treina k modelos).
- Pode ser lento para redes neurais profundas.

Comparação com Holdout

Enquanto o Holdout faz uma única divisão (ex: 70/15/15), o K-Fold repete o processo k vezes. O resultado final é a média das k avaliações, tornando a estimativa de desempenho mais estável e confiável.

Como funciona? (K-Fold)

01

Dividir em k folds

Dividir o conjunto de dados em k partes iguais (folds).

02

Treino e Validação

Em cada iteração, usar $k-1$ folds para treino e 1 fold para validação.

03

Repetir k vezes

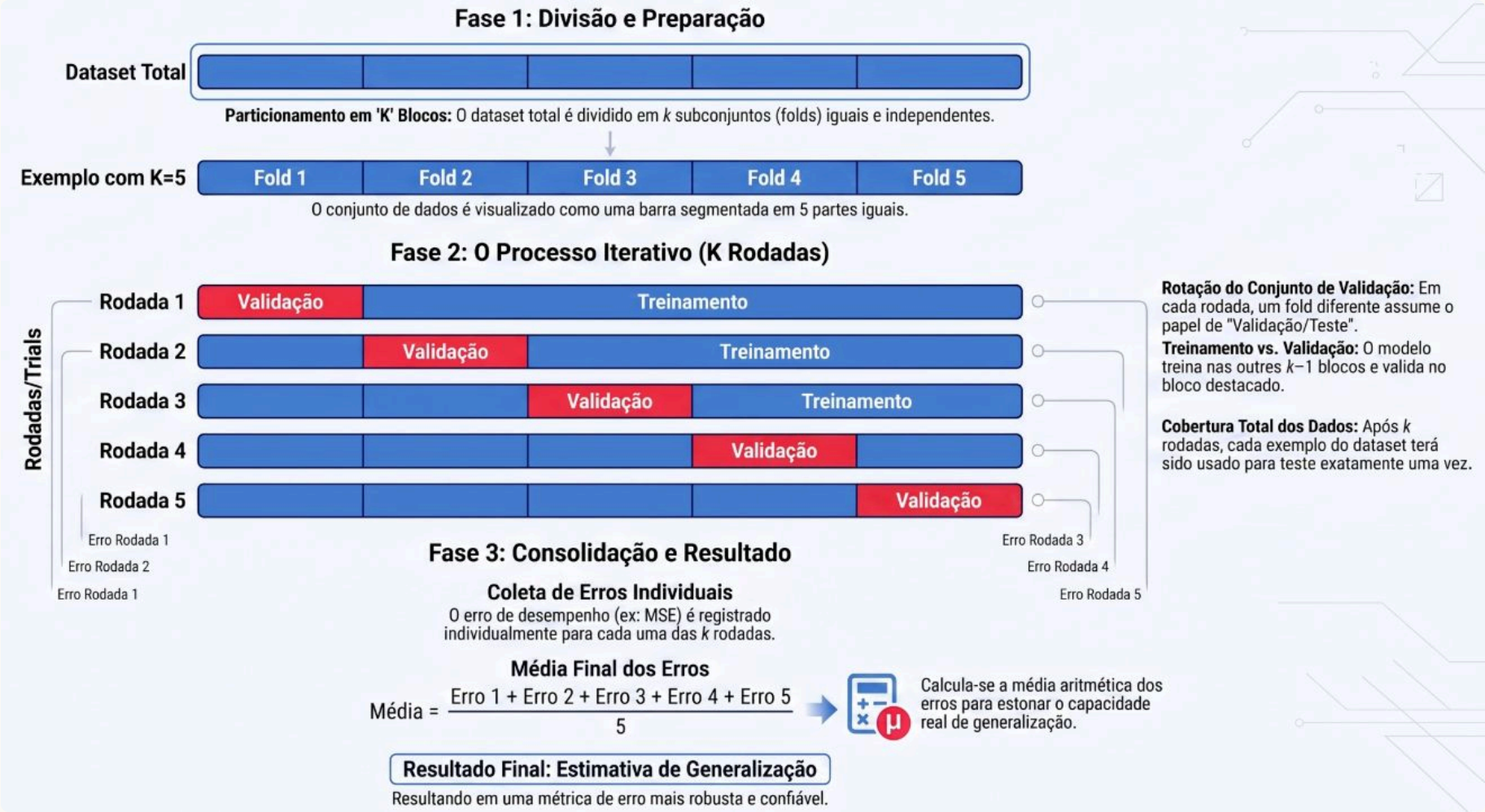
Repetir o processo k vezes, alternando o fold de validação.

04

Calcular Média

Calcular a média e o desvio padrão do desempenho entre as k iterações.

Cross-Validation (Validação Cruzada)





Teoria e Aplicações de Inteligência Computacional I

Unidade 1 – Aula 3 – Introdução aos Fundamentos de
Inteligência Computacional

Capítulo 4 – Funções de Ativação

Funções de Ativação

Sigmoide Logística: tradicional, mas sofre com a saturação (gradientes em valores extremos). Dizemos que a função está **saturada** quando pequenas variações na entrada praticamente não alteram a saída;

Tangente Hiperbólica (Tanh): frequentemente superior à sigmoide por ser simétrica em relação à origem, facilitando o treinamento;

ReLu (Unidade Linear Retificada): padrão moderno; evita a saturação para valores positivos e torna a otimização muito mais fácil.

Funções de Ativação – Problemas de Saturação da Sigmoid

1. A função Sigmoid Logística

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Transforma qualquer valor real em um número entre 0 e 1.



x muito negativo
saída ≈ 0
(saturação)

x próximo de 0
saída varia rápido
(região útil)

x muito positivo
saída ≈ 1
(saturação)

2. Saturação: pequenas variações na entrada causam quase nenhuma variação na saída

Entrada x	Saída $\sigma(x)$
-10	0,000045
-5	0,0067
0	0,5000
5	0,9933
10	0,99995

Varia muito pouco
(próximo de 0)

Varia rápido
(região útil)

Varia muito pouco
(próximo de 1)



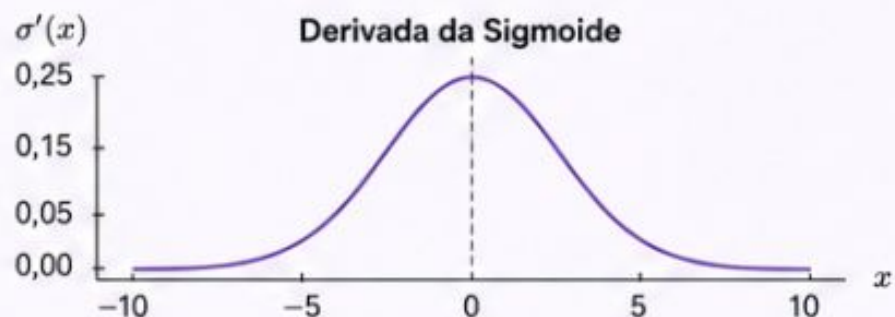
Nas extremidades, a função está "achatada". Isso é a **saturação**.

Funções de Ativação – Problemas de Saturação da Sigmoid

3. A derivada da Sigmoid

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

A derivada (inclinação) é máxima no centro e vai para zero nas extremidades.



Saída $\sigma(x)$	0,00	0,10	0,50	0,90	1,00
Derivada $\sigma'(x)$	0,00	0,09	0,25	0,09	0,00



Nas extremidades, a derivada ≈ 0 .

4. O que acontece no Backpropagation?

O gradiente é propagado da saída para a entrada.

De forma simplificada:

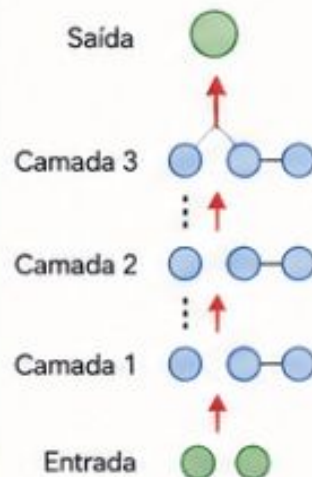
$$\delta^{(l)} = \delta^{(l+1)} \cdot W \cdot \sigma'(z)$$

Se $\sigma'(z) = 0,01$ em cada camada:

Camada de saída: 0,01
↓
Camada anterior: $0,01 \times 0,01 = 0,0001$
↓
Próxima camada: $0,0001 \times 0,01 = 0,000001$
↓
Mais abaixo: 10^{-8}
↓
... ainda mais: 10^{-10} ...



O gradiente praticamente **desaparece** (vanishing gradient).



Funções de Ativação – Problemas de Saturação da Sigmoide

5. Consequência: pesos quase não são atualizados

A atualização dos pesos é dada por:

$$w \leftarrow w - \eta \frac{\partial J}{\partial w}$$

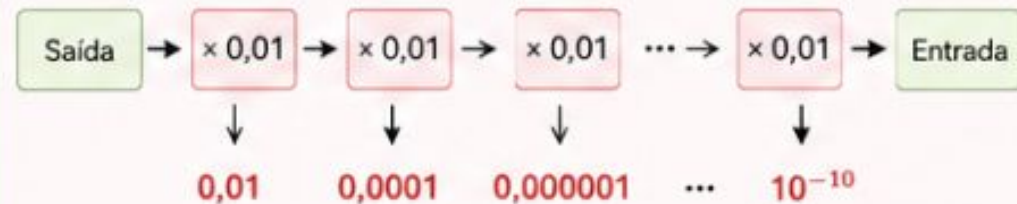
Se $\frac{\partial J}{\partial w} \approx 0 \Rightarrow \Delta w \approx 0$

Os pesos praticamente não mudam e as primeiras camadas deixam de aprender.

6. Por que é um problema em redes profundas?

Em redes com muitas camadas, o gradiente é multiplicado por várias derivadas (< 1 e, nas extremidades, ≈ 0).

Assim, ele diminui exponencialmente à medida que retropropaga, e as primeiras camadas recebem valores praticamente nulos.



Resultado: o gradiente some nas primeiras camadas e a rede "para de aprender".

Funções de Ativação – Tangente Hiperbólica

Definição Matemática

A função tangente hiperbólica é definida por:

$$\phi(v) = \tanh(v) = \frac{e^v - e^{-v}}{e^v + e^{-v}}$$

Alternativamente, pode ser expressa como uma versão reescalada e deslocada da função sigmoide logística:

$$\tanh(z) = 2\sigma(2z) - 1$$

Relação com a Sigmoide

A tanh é uma versão centralizada da sigmoide: enquanto a sigmoide produz saídas entre 0 e 1, a tanh produz saídas entre -1 e +1, com média zero. Isso torna o aprendizado mais eficiente em muitos casos.

Características

- Saída no intervalo (-1, +1).
- Simétrica em torno da origem (saída com média zero).
- Derivada: $\tanh'(v) = 1 - \tanh^2(v)$

$$\phi'(v) = 1 - \tanh^2(v)$$



Problema: Saturação

Assim como a sigmoide, a tanh também sofre com saturação nas regiões extremas ($v \rightarrow \pm\infty$), onde o gradiente se aproxima de zero, dificultando o aprendizado nas camadas mais profundas (vanishing gradient).

Funções de Ativação – ReLU (Rectified Linear Unit)

Definição Matemática

A ReLU é definida por:

$$\phi(v) = \max(0, v)$$

Sua derivada é:

$$\phi'(v) = \begin{cases} 1 & \text{se } v > 0 \\ 0 & \text{se } v \leq 0 \end{cases}$$

Por que resolve o Vanishing Gradient?

Diferente da sigmoide e da tanh, que saturam nos extremos (derivada ≈ 0), a ReLU possui derivada constante e igual a 1 para qualquer entrada positiva ($v > 0$). Isso garante que o gradiente flua sem atenuação durante o backpropagation.

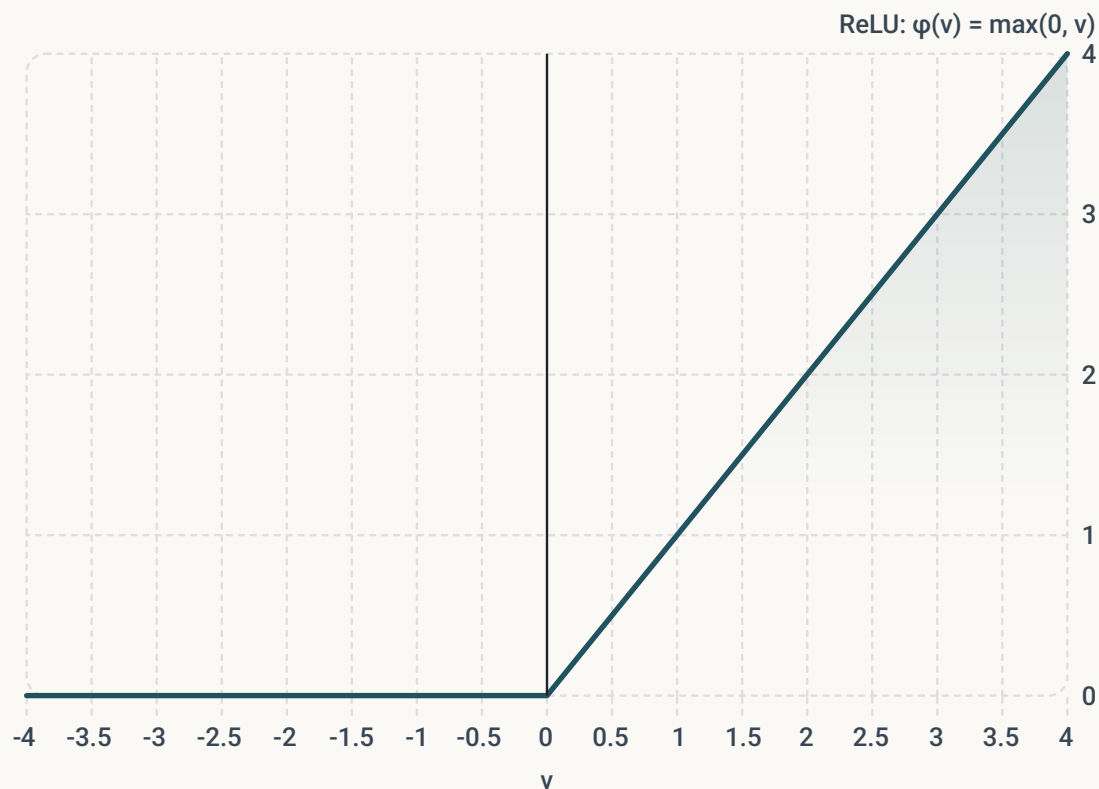
Ausência de Saturação

- Para $v > 0$: a derivada é sempre 1, o gradiente não desvanece.
- Para $v \leq 0$: o neurônio é desativado (saída = 0).
- Nas funções sigmoidais, a derivada tende a zero nos extremos, causando o vanishing gradient em redes profundas.

Vantagens

- Computacionalmente simples e eficiente.
- Acelera a convergência do treinamento.
- Evita o vanishing gradient para valores positivos.

Funções de Ativação – Gráfico da ReLU



Interpretação do Gráfico

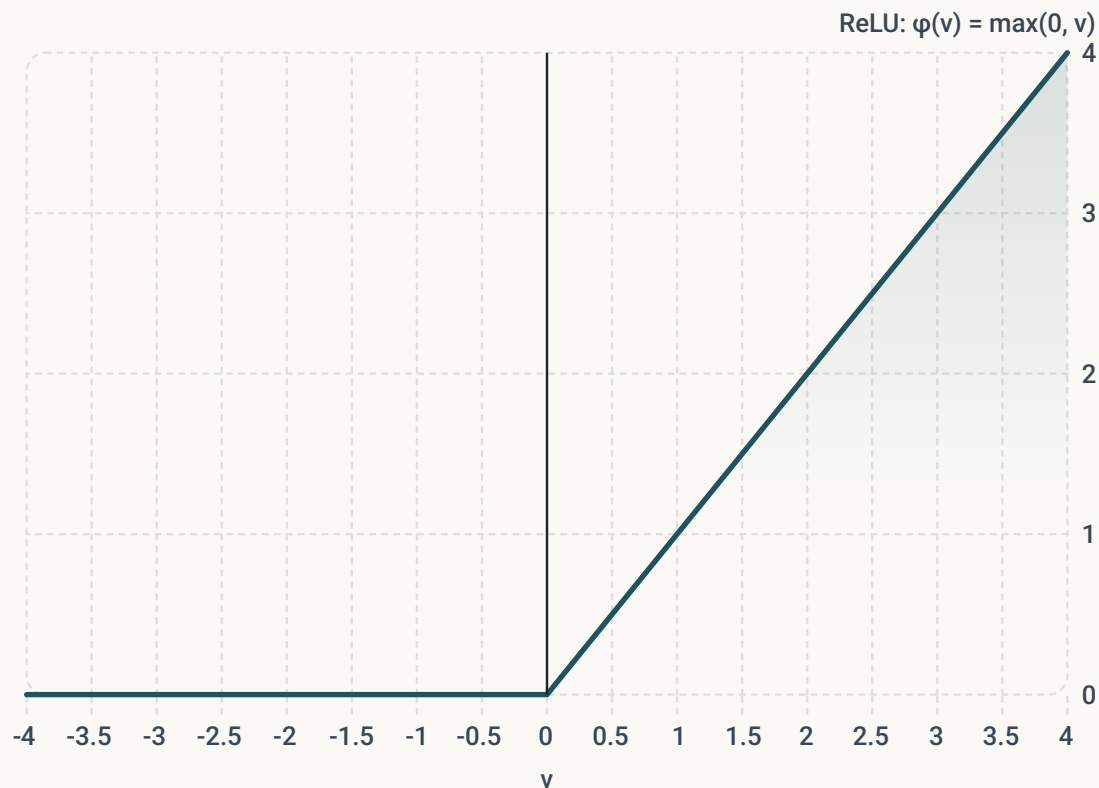
- Para $v \leq 0$: saída é sempre 0 (neurônio inativo).
- Para $v > 0$: saída cresce linearmente (neurônio ativo).
- O "cotovelo" em $v = 0$ é o ponto de ativação da função.

Derivada

$$\phi'(v) = \begin{cases} 1 & \text{se } v > 0 \\ 0 & \text{se } v \leq 0 \end{cases}$$

i Comparação com Sigmoid e Tanh: Enquanto a sigmoide e a tanh saturam nos extremos (derivada $\rightarrow 0$), a ReLU mantém derivada constante igual a 1 para $v > 0$, evitando o vanishing gradient.

Funções de Ativação – Gráfico da ReLU



Derivada

$$\phi'(v) = \begin{cases} 1 & \text{se } v > 0 \\ 0 & \text{se } v \leq 0 \end{cases}$$

- **Problema dos "Neurônios Mortos":** Devido ao gradiente nulo para entradas negativas, uma unidade ReLU pode ficar "presa" em um estado inativo. Se um neurônio for inicializado ou atualizado de tal forma que sua saída seja sempre zero para o conjunto de treinamento, ele **não receberá gradiente** e seus pesos nunca mais serão ajustados, tornando-o inútil para o aprendizado

Funções de Ativação – Leaky ReLU

Definição Matemática

A Leaky ReLU é uma generalização da ReLU que introduz uma pequena inclinação α para valores negativos:

$$h_i = \max(0, z_i) + \alpha_i \min(0, z_i)$$

Equivalentemente:

$$\phi(v) = \begin{cases} v & \text{se } v > 0 \\ \alpha v & \text{se } v \leq 0 \end{cases}$$

onde α é um valor pequeno, tipicamente 0,01.

Gradientes

- Para $z > 0$: gradiente = 1 (mantém a eficiência da ReLU padrão).
- Para $z < 0$: gradiente = α (pequeno, mas não nulo).

$$\phi'(v) = \begin{cases} 1 & \text{se } v > 0 \\ \alpha & \text{se } v \leq 0 \end{cases}$$

Leaky ReLU: Vantagens e Comparação

Por que usar a Leaky ReLU?

A ReLU padrão desativa completamente neurônios com entradas negativas (gradiente = 0), impedindo que eles aprendam — o chamado problema do Dying ReLU. A Leaky ReLU resolve isso mantendo um gradiente não nulo (α) para entradas negativas, permitindo que esses neurônios continuem a ajustar seus pesos via backpropagation.

Vantagens

- Evita o problema do Dying ReLU.
- Gradiente não nulo em todo o domínio.
- Mantém a eficiência computacional da ReLU.



Comparação ReLU vs Leaky ReLU

Na ReLU, neurônios com $z \leq 0$ ficam permanentemente inativos. Na Leaky ReLU, esses neurônios ainda recebem um gradiente $\alpha \approx 0,01$, garantindo que continuem aprendendo durante o treinamento.

Funções de Ativação – Função Linear (Identidade)

Definição Matemática

A função linear, também chamada de função identidade, simplesmente repassa o valor de entrada como saída, sem transformação:

$$\phi(v) = v$$

Sua derivada é constante:

$$\phi'(v) = 1$$

Quando usar?

Utilizada exclusivamente na camada de saída em tarefas de regressão, quando o objetivo é prever um valor contínuo sem restrição de intervalo (ex: temperatura, preço, carga elétrica).

Características

- Saída no intervalo $(-\infty, +\infty)$.
- Derivada constante igual a 1 – sem saturação.
- Não introduz não-linearidade na rede.



Por que não usar em camadas ocultas?

A função linear não introduz não-linearidade. Se todas as camadas usassem ativação linear, a rede inteira se comportaria como um único neurônio linear, perdendo a capacidade de aprender funções complexas. Por isso, ela é reservada apenas para a camada de saída em regressão.

Escolha da Função de Ativação – Camadas Ocultas

A escolha da função de ativação é uma das decisões de design mais críticas em Redes Neurais Profundas, pois introduz a não-linearidade necessária para aprender funções complexas e afeta diretamente a facilidade de otimização do modelo.

ReLU (Recomendação Padrão)

É a recomendação padrão para a maioria das redes feedforward modernas. Computacionalmente eficiente, sua derivada é constante ($= 1$) para valores positivos, evitando o vanishing gradient.

Leaky ReLU / PReLU / Maxout

Generalizações da ReLU. Usar quando se deseja evitar "neurônios mortos" — unidades que nunca ativam e não recebem gradiente. Garantem gradiente não nulo mesmo para entradas negativas.

Tangente Hiperbólica (Tanh)

Produz saídas no intervalo $[-1, +1]$. Preferida à sigmoide em camadas ocultas por ser centrada em zero, facilitando o treinamento de redes profundas. É também a ativação padrão em RNNs.

Escolha da Função de Ativação – Camada de Saída

A função na camada de saída é escolhida estritamente de acordo com o tipo de tarefa que o modelo deve realizar.

Linear (Identidade)

Tarefa: Regressão. Uso: Quando o objetivo é prever um valor contínuo (ex: temperatura, carga elétrica).

Frequentemente associada a distribuições Gaussianas, onde a rede prevê a média da distribuição.

Sigmoide Logística

Tarefa: Classificação Binária. Uso: Quando a saída deve representar a probabilidade de um exemplo pertencer a uma classe (valor entre 0 e 1). É a base da regressão logística.

Softmax

Tarefa: Classificação Multiclasse. Uso: Quando o modelo deve escolher uma entre n classes mutuamente exclusivas. Garante que a soma de todas as saídas seja igual a 1, formando uma distribuição de probabilidade válida.

Softplus

Tarefa: Estimativa de parâmetros estritamente positivos. Uso: Comumente usada para prever a variância ou desvio padrão em modelos probabilísticos, garantindo que esses valores nunca sejam negativos ou zero.

Escolha da Função de Ativação – Resumo

Camada	Função Recomendada	Tarefa / Motivo Principal
Oculto	ReLU (ou variantes)	Facilita a otimização e evita gradientes desvanecentes
Oculto	Leaky ReLU / PReLU	Evita neurônios mortos (entradas negativas)
Oculto	Tanh	Ativações centradas em zero; padrão em RNNs
Oculto	Sigmoide	Desencorajada — sofre com vanishing gradient
Saída	Linear	Regressão — previsão de valores contínuos
Saída	Sigmoide	Classificação Binária — saída como probabilidade
Saída	Softmax	Classificação Multiclasse — distribuição de probabilidade
Saída	Softplus	Estimativa de parâmetros estritamente positivos



5

Teoria e Aplicações de Inteligência Computacional I

Unidade 1 – Aula 3 – Introdução aos Fundamentos de
Inteligência Computacional

Capítulo 5 – Implementações

Importação das Bibliotecas

```
# Importação das Bibliotecas
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
import matplotlib.pyplot as plt
```

torch: Biblioteca principal do PyTorch para operações com tensores e computação em GPU.

torch.nn: Módulo para construção de redes neurais (camadas, funções de ativação, etc.).

torch.optim: Algoritmos de otimização (SGD, Adam, etc.) para atualização dos pesos.

TensorDataset / DataLoader: Utilitários para organizar e iterar sobre os dados em batches.

matplotlib.pyplot: Biblioteca para visualização de gráficos e resultados do treinamento.

Hiperparâmetros (Implementação do problema do XOR)

```
LEARNING_RATE = 0.001
EPOCHS = 10000
BATCH_SIZE = 1 # batch_size: # 1 = Gradiente Descendente Estocástico (SGD), # 2 = Mini-Batch, # 4 = Batch Gradient Descente
HIDDEN_LAYERS = 1
NEURONS = 3
ACTIVATION = "relu" # relu | sigmoid | tanh
OPTIMIZER = "SGD" # SGD | Adam
MOMENTUM = 0.8
WEIGHT_DECAY = 0.0001
DROPOUT = 0
EARLY_STOPPING = True
PATIENCE = 100
```

Parâmetros de Treinamento

- LEARNING_RATE** ($\eta = 0.001$): Taxa de aprendizado — controla o tamanho do passo na descida do gradiente.
- EPOCHS** (10000): Número máximo de épocas de treinamento.
- BATCH_SIZE** (1): Define o modo de gradiente descendente. 1 = SGD, valores intermediários = Mini-Batch, total = Batch GD.
- OPTIMIZER** (SGD | Adam): Algoritmo de otimização utilizado para atualizar os pesos.
- MOMENTUM** (0.8): Termo de momento — acumula gradientes passados para acelerar a convergência.

Parâmetros de Arquitetura e Regularização

- HIDDEN_LAYERS** (1): Número de camadas ocultas da rede.
- NEURONS** (3): Número de neurônios por camada oculta.
- ACTIVATION** (relu | sigmoid | tanh): Função de ativação das camadas ocultas.
- WEIGHT_DECAY** (0.0001): Penalização L2 sobre os pesos para evitar overfitting.
- DROPOUT** (0): Fração de neurônios desativados aleatoriamente durante o treinamento.
- EARLY_STOPPING + PATIENCE** (100): Interrompe o treinamento se a perda não melhorar por 100 épocas consecutivas, evitando overfitting.

Base de Dados XOR

```
# BASE XOR
```

```
X = torch.tensor([  
    [0., 0.],  
    [0., 1.],  
    [1., 0.],  
    [1., 1.]  
])
```

```
y = torch.tensor([  
    [0.],  
    [1.],  
    [1.],  
    [0.]  
])
```

Criando o Dataset e DataLoader

```
dataset = TensorDataset(X, y)

loader = DataLoader(dataset,
                    batch_size=BATCH_SIZE,
                    shuffle=True)
```

TensorDataset

Agrupar os tensores X (entradas) e y (saídas) em um único objeto de dataset. Isso facilita o acesso às amostras de forma indexada, mantendo a correspondência entre entrada e saída.

Por que usar?

- Organiza os dados em pares (X, y) prontos para iteração.
- Compatível com o DataLoader do PyTorch.
- Simplifica o pipeline de treinamento.

Próximo passo: DataLoader

O DataLoader será responsável por:

- Iterar sobre o dataset em batches (definido por BATCH_SIZE).
- Embaralhar os dados a cada época (shuffle=True).
- Alimentar a rede durante o treinamento de forma eficiente.

 O BATCH_SIZE definido nos hiperparâmetros controla quantas amostras são processadas por vez: BATCH_SIZE=1 → SGD, BATCH_SIZE intermediário → Mini-Batch, BATCH_SIZE=total → Batch Gradient Descent.

Escolha da Função de Ativação (Código)

```
# ESCOLHA DA FUNÇÃO DE ATIVAÇÃO

if ACTIVATION.lower() == "relu":
    activation = nn.ReLU()

elif ACTIVATION.lower() == "sigmoid":
    activation = nn.Sigmoid()

elif ACTIVATION.lower() == "tanh":
    activation = nn.Tanh()

else:
    raise ValueError("Função de ativação inválida.")
```

Boas práticas

Centralizar a escolha da função de ativação em um único bloco condicional facilita a experimentação — basta alterar o valor de `ACTIVATION` nos hiperparâmetros para testar diferentes configurações sem modificar a arquitetura da rede.

Construção da Rede Neural (MLP) – Primeiras Camadas

```
# CONSTRUÇÃO DA REDE NEURAL (MLP) - Primeiras Camadas
```

```
layers = [] # lista para armazenamento das camadas
```

```
# Primeira camada
```

```
layers.append(nn.Linear(2, NEURONS))
```

```
layers.append(activation)
```

layers = []

Lista Python que armazenará sequencialmente todas as camadas da rede. Cada elemento adicionado representa uma transformação aplicada ao sinal durante o forward pass.

nn.Linear(2, NEURONS)

Implementa uma transformação linear (afim) entre a entrada e a saída:

$$y = xW + b$$

Onde:

- $x \rightarrow$ vetor de entrada
- $W \rightarrow$ matriz de pesos
- $b \rightarrow$ vetor de bias
- $y \rightarrow$ saída da camada

O valor 2 indica que a camada recebe 2 entradas (as duas entradas do XOR: x_1 e x_2). NEURONS define o número de neurônios na camada oculta.

Por que "Linear"?

O nome vem do fato de que a camada implementa exatamente uma transformação linear (afim) entre entrada e saída. Apesar do nome, a não-linearidade é introduzida pela função de ativação adicionada logo após.

layers.append(activation)

Adiciona a função de ativação escolhida (ReLU, Sigmoid ou Tanh) após a camada linear. Essa sequência – camada linear seguida de ativação – é o bloco fundamental de qualquer MLP.



Fluxo até agora

layers[0] = nn.Linear(2, NEURONS) $\rightarrow$ transformação linear

layers[1] = activation $\rightarrow$ não-linearidade

Construção da Rede Neural (MLP) – Dropout

```
# Dropout (opcional)
if DROPOUT > 0:
    layers.append(nn.Dropout(DROPOUT))
# Camadas ocultas adicionais
for _ in range(HIDDEN_LAYERS - 1):
    layers.append(nn.Linear(NEURONS, NEURONS))
    layers.append(activation)
    if DROPOUT > 0:
        layers.append(nn.Dropout(DROPOUT))
```

nn.Dropout(DROPOUT)

O Dropout desativa aleatoriamente uma fração dos neurônios durante o treinamento, definida pelo hiperparâmetro DROPOUT (ex: 0.2 = 20% dos neurônios desativados por passo). É aplicado apenas se DROPOUT > 0, tornando-o opcional.

Por que usar Dropout?

- Evita que a rede dependa de neurônios específicos.
- Reduz o overfitting, forçando a rede a aprender representações mais robustas.
- Durante a inferência (teste), o Dropout é automaticamente desativado pelo PyTorch.

Construção da Rede Neural (MLP) – Camadas Ocultas Adicionais

Como funciona o laço?

O `for _ in range(HIDDEN_LAYERS - 1)` adiciona as camadas ocultas restantes. A primeira camada já foi adicionada anteriormente, por isso o intervalo começa em `HIDDEN_LAYERS - 1`.

Cada iteração adiciona:

1. `nn.Linear(NEURONS, NEURONS)` → transformação linear entre neurônios da mesma largura
2. `activation` → função de ativação escolhida
3. `nn.Dropout` (se `DROPOUT > 0`) → regularização opcional

Fluxo das camadas

Para `HIDDEN_LAYERS=2` e `DROPOUT=0.2`:

```
layers = [Linear(2, N), activation, Dropout,  
          Linear(N, N), activation, Dropout]
```

`nn.Linear(NEURONS, NEURONS)`

Nas camadas ocultas intermediárias, tanto a entrada quanto a saída têm o mesmo número de neurônios (`NEURONS`), mantendo a largura da rede constante ao longo das camadas ocultas.

Construção da Rede Neural (MLP) – Camada de Saída

```
# Camada de saída  
layers.append(nn.Linear(NEURONS, 1))  
layers.append(nn.Sigmoid())
```

nn.Linear(NEURONS, 1)

A camada de saída recebe **NEURONS** entradas (vindas da última camada oculta) e produz exatamente 1 saída — o valor previsto pela rede. Para o problema XOR, essa saída representa a probabilidade de a entrada pertencer à classe 1.

Por que 1 saída?

O XOR é um problema de classificação binária: a resposta é sempre 0 ou 1. Uma única saída é suficiente para representar essa probabilidade.

nn.Sigmoid()

A Sigmoid é usada na camada de saída pois comprime qualquer valor real para o intervalo (0, 1), tornando a saída interpretável como uma probabilidade:

$$\sigma(v) = \frac{1}{1 + e^{-v}}$$

- Saída próxima de 1 → rede prevê classe 1
- Saída próxima de 0 → rede prevê classe 0

Construção da Rede Neural (MLP) – nn.Sequential

```
# Cria a rede neural
model = nn.Sequential(*layers)

# Mostra a arquitetura da rede
print(model)
```

nn.Sequential(*layers)

O `nn.Sequential` encapsula a lista `layers` em um modelo PyTorch. O operador `*` (unpacking) desempacota a lista, passando cada camada como argumento separado. O resultado é um modelo que aplica as camadas em sequência, uma após a outra, durante o forward pass.

Por que nn.Sequential?

- Organiza todas as camadas em um único objeto de modelo.
- Permite chamar `model(X)` para executar o forward pass completo automaticamente.
- Compatível com todos os otimizadores e funções de perda do PyTorch.

print(model)

Exibe a arquitetura completa da rede no console. Para `HIDDEN_LAYERS=1`, `NEURONS=3` e `ACTIVATION="relu"`, a saída seria:

```
Sequential(
  (0): Linear(in_features=2, out_features=3, bias=True)
  (1): ReLU()
  (2): Linear(in_features=3, out_features=1, bias=True)
  (3): Sigmoid()
)
```

Função de Perda (Loss Function)

```
# FUNÇÃO DE PERDA  
criterion = nn.MSELoss()
```

nn.MSELoss()

O MSE (Mean Squared Error) é a função de perda utilizada para medir o erro entre a saída prevista pela rede e o valor real esperado. É calculado como a média dos quadrados das diferenças:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Onde:

- $\hat{y}_i$ → saída prevista pela rede
- y_i → valor real esperado
- n → número de amostras

Por que MSE?

- Penaliza erros grandes de forma quadrática, tornando o treinamento mais sensível a previsões muito erradas.
- É diferenciável em todo o domínio, permitindo o cálculo do gradiente via backpropagation.
- Adequado para problemas de regressão e classificação binária com saída contínua (como o XOR com Sigmoides).

Escolha do Otimizador – SGD

```
if OPTIMIZER.upper() == "SGD":  
    optimizer = optim.SGD(  
        model.parameters(),    # Parâmetros (pesos e bias) da rede  
        lr=LEARNING_RATE,      # Taxa de aprendizagem  
        momentum=MOMENTUM,      # Acelera a convergência  
        weight_decay=WEIGHT_DECAY # Regularização L2  
    )
```

optim.SGD

O SGD (Stochastic Gradient Descent) é o otimizador clássico. Atualiza os pesos na direção oposta ao gradiente:

$$w \leftarrow w - \eta \nabla J(w)$$

Parâmetros configurados

- **model.parameters()**: passa todos os pesos e bias da rede para o otimizador.
- **lr** (LEARNING_RATE): tamanho do passo de atualização.
- **momentum** (MOMENTUM): acumula gradientes passados para acelerar a convergência e suavizar oscilações.
- **weight_decay** (WEIGHT_DECAY): aplica regularização L2, penalizando pesos grandes.

Escolha do Otimizador – Adam

`optim.Adam`

O Adam (Adaptive Moment Estimation) adapta a taxa de aprendizado individualmente para cada parâmetro, combinando as vantagens do SGD com momento e do RMSProp. É mais robusto e geralmente converge mais rápido que o SGD puro.

Parâmetros configurados:

- **`model.parameters()`**: passa todos os pesos e bias da rede.
- **`lr`** (`LEARNING_RATE`): taxa de aprendizado base.
- **`weight_decay`** (`WEIGHT_DECAY`): regularização L2.



Caso o valor de `OPTIMIZER` não seja "SGD" nem "ADAM", um `ValueError` é lançado, interrompendo a execução antes do treinamento.

SGD vs Adam

- SGD: mais simples, requer ajuste fino do momentum e lr.
- Adam: adapta o lr automaticamente, mais robusto para a maioria dos problemas.

Loop de Treinamento – Variáveis de Controle

```
# Lista utilizada para armazenar o valor da Loss em cada época
loss_history = []

# Variáveis utilizadas pelo Early Stopping
best_loss = float("inf")
counter = 0

print("Iniciando treinamento...")
```

loss_history = []

Lista que armazena o valor da Loss a cada época. Será usada ao final para plotar o gráfico de evolução do treinamento com matplotlib.

best_loss e counter

Variáveis de controle do Early Stopping:

- **best_loss**: armazena a menor Loss já observada. Inicializada com `float("inf")` para garantir que qualquer Loss real seja menor na primeira comparação.
- **counter**: conta quantas épocas consecutivas sem melhora na Loss.

Loop de Treinamento – Início do Loop

```
# Loop principal de treinamento
for epoch in range(EPOCHS):

    # Coloca a rede em modo de treinamento
    model.train()

    # Soma da Loss da época
    epoch_loss = 0
```

for epoch in range(EPOCHS)

Loop principal que itera por até EPOCHS épocas. Em cada iteração, a rede processa todos os batches do DataLoader e atualiza os pesos via backpropagation.

model.train()

Coloca a rede em modo de treinamento. Isso ativa camadas como Dropout e BatchNorm, que se comportam de forma diferente durante treino e inferência (model.eval()).

epoch_loss = 0

Acumulador zerado a cada época para somar a Loss de todos os batches. Ao final da época, será dividido pelo número de batches para obter a Loss média.

i Fluxo da época: Para cada época → model.train() → itera sobre batches → zero_grad() → forward → loss → backward → step → calcula Loss média → Early Stopping.

Loop de Treinamento – Iteração sobre Batches

```
# Percorre todos os batches do DataLoader
for batch_x, batch_y in loader:
    # 1) Zera os gradientes calculados anteriormente
    optimizer.zero_grad()
```

for batch_x, batch_y in loader

Itera sobre os batches do DataLoader. A cada iteração:

- **batch_x**: contém as entradas do batch atual (ex: `[[0., 0.]]` para `BATCH_SIZE=1`).
- **batch_y**: contém os valores esperados correspondentes (ex: `[[0.]]`).

optimizer.zero_grad()

Zera os gradientes acumulados do passo anterior. No PyTorch, os gradientes são acumulados por padrão — é obrigatório zerá-los antes de cada novo batch para evitar que gradientes de iterações anteriores contaminem o cálculo atual.

⚠ Esquecer o `zero_grad()` é um erro comum em PyTorch. Sem ele, os gradientes se acumulam incorretamente ao longo dos batches, corrompendo a atualização dos pesos.

Loop de Treinamento – Forward Pass e Cálculo da Loss

```
# 2) Forward Pass
# Calcula a saída da rede neural
prediction = model(batch_x)
# 3) Calcula a função de perda (Loss)
loss = criterion(prediction, batch_y)
```

prediction = model(batch_x)

Executa o forward pass: os dados de entrada percorrem todas as camadas da rede em sequência (Linear → Ativação → ... → Sigmoid) e produzem a saída prevista. O PyTorch executa isso automaticamente com o `nn.Sequential`.

loss = criterion(prediction, batch_y)

Calcula o MSE entre a saída prevista (`prediction`) e o valor real esperado (`batch_y`). Esse valor representa o erro da rede naquele batch e será usado pelo backpropagation para calcular os gradientes.

- ❗ Esses dois passos – forward pass e cálculo da loss – são a base da avaliação do erro. O próximo passo é o backpropagation (`loss.backward()`), que usa esse erro para calcular os gradientes de todos os pesos.

Loop de Treinamento – Backpropagation

```
# 4) Backpropagation  
# Calcula automaticamente todos os gradientes  
# utilizando Autograd.  
loss.backward()
```

loss.backward()

Executa o Backpropagation automaticamente via Autograd do PyTorch. Calcula o gradiente da Loss em relação a todos os pesos e bias da rede, aplicando a regra da cadeia camada por camada — da saída até a entrada.

Como o Autograd funciona?

O PyTorch constrói um grafo computacional durante o forward pass. O backward() percorre esse grafo no sentido inverso, calculando automaticamente os gradientes de cada parâmetro usando a regra da cadeia.

- ❗ Após o `loss.backward()`, cada parâmetro da rede possui seu gradiente armazenado em `param.grad`. O próximo passo é o `optimizer.step()`, que usa esses gradientes para atualizar os pesos.

Loop de Treinamento – Atualização dos Pesos e Loss da Época

```
# 5) Atualização dos pesos
# O otimizador utiliza os gradientes calculados
# pelo Backpropagation para atualizar todos os
# pesos e bias da rede.
optimizer.step()
# Soma a Loss do batch atual
epoch_loss += loss.item()
# Calcula a Loss média da época
epoch_loss /= len(loader)
# Armazena para construção do gráfico
loss_history.append(epoch_loss)
# Mostra a evolução do treinamento
if (epoch + 1) % 500 == 0:
    print(f"Época: {epoch+1:5d} | Loss = {epoch_loss:.6f}" + "\n")
```

optimizer.step()

Usa os gradientes calculados pelo `backward()` para atualizar todos os pesos e bias da rede, de acordo com o algoritmo do otimizador escolhido (SGD ou Adam).

epoch_loss += loss.item()

Acumula a Loss de cada batch. O método `.item()` converte o tensor PyTorch em um número Python simples.

epoch_loss /= len(loader)

Calcula a Loss média da época dividindo o total acumulado pelo número de batches.

loss_history.append(epoch_loss)

Armazena a Loss média para plotar o gráfico de evolução ao final do treinamento.

if (epoch + 1) % 500 == 0

Exibe o progresso a cada 500 épocas para monitorar o treinamento sem sobrecarregar o console.

Loop de Treinamento – Early Stopping

```
# EARLY STOPPING
# Se a Loss não melhorar durante PATIENCE épocas
# consecutivas, o treinamento é interrompido.
if EARLY_STOPPING:
    if epoch_loss < best_loss:
        best_loss = epoch_loss
        counter = 0
    else:
        counter += 1
```

Como funciona?

A cada época, a Loss atual é comparada com a melhor Loss já registrada (best_loss):

- Se melhorou: best_loss é atualizado e counter é zerado.
- Se não melhorou: counter é incrementado em 1.

Por que usar Early Stopping?

Evita o overfitting ao interromper o treinamento quando a rede para de melhorar, economizando tempo computacional e preservando o melhor estado do modelo.

 O parâmetro **PATIENCE (= 100)** define quantas épocas consecutivas sem melhora são toleradas. O critério de parada será verificado no próximo passo.

Loop de Treinamento – Early Stopping: Critério de Parada

```
if counter >= PATIENCE:  
    print("Early Stopping ativado.")  
    print(f"Treinamento encerrado na época {epoch+1}.")  
    break
```

if counter >= PATIENCE

Quando o contador de épocas sem melhora atinge o valor de PATIENCE (= 100), o treinamento é interrompido imediatamente com break, saindo do loop principal de épocas.

break

Encerra o loop `for epoch in range(EPOCHS)` antes de atingir o número máximo de épocas. O modelo é preservado no estado em que estava ao ser interrompido.

Fluxo completo do Early Stopping

1. $\text{epoch_loss} < \text{best_loss}$? → Sim: zera counter, atualiza `best_loss`.
2. $\text{epoch_loss} \geq \text{best_loss}$? → Não: incrementa counter.
3. $\text{counter} \geq \text{PATIENCE}$? → Sim: imprime mensagem e encerra com break.

Teste da Rede Neural – Configuração

```
# TESTE DA REDE
print("RESULTADOS")
# Coloca a rede em modo de avaliação
model.eval()
# Desabilita o cálculo de gradientes
with torch.no_grad():
    output = model(X)
```

model.eval()

Coloca a rede em modo de avaliação. Isso desativa camadas como Dropout e BatchNorm, que se comportam de forma diferente durante o treinamento. É obrigatório chamar `model.eval()` antes de qualquer inferência.

torch.no_grad()

Desabilita o cálculo de gradientes dentro do bloco. Como não estamos treinando, não precisamos do grafo computacional – isso economiza memória e acelera a inferência.

output = model(X)

Executa o forward pass com todas as 4 amostras do XOR de uma vez, retornando as previsões da rede para cada entrada.

Teste da Rede Neural – Exibição dos Resultados

```
print("\nEntrada -> Saída Esperada | Saída Predita")
for entrada, esperado, previsto in zip(X, y, output):
    if previsto.item() >= 0.5:
        classe = 1
    else:
        classe = 0
    print(f"{entrada.numpy()} -> {int(esperado.item())} | {previsto.item():.4f} ({classe})")
```

`zip(X, y, output)`

Itera simultaneamente sobre as entradas (X), os valores esperados (y) e as previsões da rede (output), agrupando os três em tuplas a cada iteração.

`previsto.item() >= 0.5`

Aplica o limiar de decisão: como a saída da Sigmoid está entre 0 e 1, valores ≥ 0.5 são classificados como classe 1, e valores < 0.5 como classe 0.

Saída esperada do XOR:

- $[0, 0] \rightarrow 0$ | previsão ≈ 0.0
- $[0, 1] \rightarrow 1$ | previsão ≈ 1.0
- $[1, 0] \rightarrow 1$ | previsão ≈ 1.0
- $[1, 1] \rightarrow 0$ | previsão ≈ 0.0

i O formato de saída `{previsto.item():.4f}` exibe o valor contínuo da Sigmoid com 4 casas decimais, seguido da classe binária (0 ou 1) entre parênteses.

Gráfico da Curva de Aprendizado

```
# GRÁFICO DA CURVA DE APRENDIZADO
plt.figure(figsize=(8, 5))
plt.plot(loss_history, linewidth=2)
plt.title("Curva de Aprendizado")
plt.xlabel("Épocas")
plt.ylabel("Loss (MSE)")
plt.grid(True)
plt.show()
```

plt.figure(figsize=(8, 5))

Cria uma nova figura com dimensões 8×5 polegadas, definindo o tamanho do gráfico exibido.

plt.plot(loss_history, linewidth=2)

Plota a lista `loss_history` no eixo Y, onde cada índice representa uma época. O parâmetro `linewidth=2` deixa a linha mais visível.

plt.grid(True)

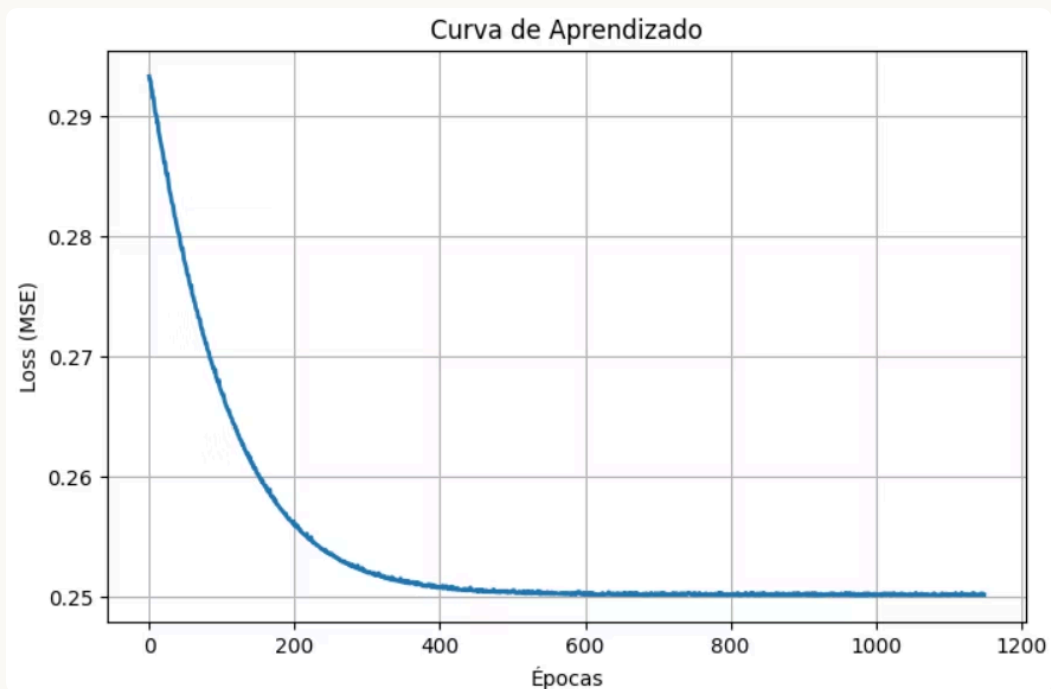
Adiciona uma grade ao gráfico, facilitando a leitura dos valores de Loss ao longo das épocas.

O que a Curva de Aprendizado mostra?

- Uma curva decrescente indica que a rede está aprendendo corretamente.
- Uma curva que para de cair indica convergência ou Early Stopping ativado.
- Uma curva instável (oscilações) pode indicar taxa de aprendizado muito alta.

i A Curva de Aprendizado é uma das ferramentas mais importantes para diagnosticar o treinamento: permite identificar overfitting, underfitting e problemas de convergência.

Resultado: Curva de Aprendizado



Interpretação do Gráfico

- A curva começa em Loss ≈ 0.293 e decresce rapidamente nas primeiras épocas.
- A partir da época ~ 400 , a curva se estabiliza em torno de Loss ≈ 0.250 .
- O treinamento foi encerrado por volta da época 1200, indicando que o Early Stopping foi ativado ou o número máximo de épocas foi atingido.

Referências Bibliográficas

- HAYKIN, S. Neural Networks and Learning Machines. 3ª ed. Pearson.
- GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. Deep Learning. MIT Press, 2016.
- MITCHELL, T. Machine Learning. McGraw-Hill, 1997.
- BISHOP, C. Pattern Recognition and Machine Learning. Springer, 2006.

Material baseado na apostila: Inteligência Computacional – Fundamentos, Algoritmos e Aplicações em Engenharia Elétrica (Versão Editorial 2.0) – Prof. Dr. Guilherme Augusto Defalque – PPGEE/UFMS